SPRING

CHEAT SHEETS

A Complete Reference for Spring Boot & the Spring Framework

IoC & DI · Configuration · Auto-Config · REST APIs · Problem Details · API Versioning · HTTP Clients · Spring Data JPA · Security · Testing · Actuator · Native Images

67 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01–05	Introduction & Setup
06–12	Core Container: IoC & Dependency Injection
13–17	Configuration & Auto-Configuration
18–19	Aspect-Oriented Programming
20–27	Spring MVC: Building REST APIs

WEB & DATA

28–29	Server-Side Views
30–32	HTTP Clients
33–35	Data Access: JDBC & Schema
36–44	Spring Data JPA
45–51	Spring Security

PRODUCTION

52–56	Testing
57–59	Actuator & Observability
60–65	Advanced Topics
66–67	Packaging & Deployment

SP

Table of Contents

67 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & Ecosyst…
- 02 Creating a Project
- 03 Starters & Dependenc…
- 04 Application Structure
- 05 Running, Building & De…

CORE CONTAINER: IOC & DEPENDENCY INJECTION 06 – 12

- 06 The IoC Container
- 07 Stereotypes & Compo…
- 08 Dependency Injection
- 09 @Configuration & @B…
- 10 Resolving Ambiguity
- 11 Bean Scopes
- 12 Bean Lifecycle & Callb…

CONFIGURATION & AUTO-CONFIGURATION 13 – 17

- 13 application.properties …
- 14 @Value & Environment
- 15 @ConfigurationProper…
- 16 Profiles
- 17 Auto-Configuration & …

ASPECT-ORIENTED PROGRAMMING 18 – 19

- 18 AOP Concepts & Proxi…
- 19 Advice & Pointcut Expr…

SPRING MVC: BUILDING REST APIS 20 – 27

- 20 Controllers & Request …
- 21 Reading Request Data
- 22 Building Responses
- 23 Bean Validation
- 24 Exception Handling
- 25 JSON & Jackson
- 26 API Versioning
- 27 Filters, Interceptors & …

SERVER-SIDE VIEWS 28 – 29

- 28 Thymeleaf Basics
- 29 Thymeleaf Forms & St…

HTTP CLIENTS 30 – 32

- 30 RestClient
- 31 HTTP Interface Clients
- 32 WebClient

DATA ACCESS: JDBC & SCHEMA 33 – 35

- 33 DataSource Configurat…
- 34 JdbcClient & JdbcTem…
- 35 Schema Init & Migratio…

SPRING DATA JPA 36 – 44

- 36 JPA Entities
- 37 Entity Relationships
- 38 Repositories
- 39 Derived Query Methods
- 40 @Query & Projections
- 41 Paging & Sorting
- 42 Transactions
- 43 Fetching & N+1 Proble…
- 44 Auditing & Specificatio…

SPRING SECURITY 45 – 51

- 45 Security Architecture
- 46 Configuring HttpSecuri…
- 47 Users & Password Enc…
- 48 Method Security
- 49 Form Login, Sessions &…
- 50 JWT Resource Server
- 51 OAuth2 Login & Client

TESTING 52 – 56

- 52 @SpringBootTest
- 53 Test Slices
- 54 MockMvc & RestTestCl…
- 55 Mocking Beans
- 56 Testcontainers

ACTUATOR & OBSERVABILITY 57 – 59

- 57 Actuator Endpoints
- 58 Metrics & Tracing
- 59 Logging

ADVANCED TOPICS 60 – 65

- 60 Application Events
- 61 Scheduling
- 62 @Async & Virtual Thre…
- 63 Caching
- 64 Retry & Resilience
- 65 WebFlux & Reactive Ba…

PACKAGING & DEPLOYMENT 66 – 67

- 66 Packaging & Containers
- 67 GraalVM Native Images

Introduction & Ecosystem

WHAT SPRING IS

Spring Framework (2003) is a Java framework built on **Inversion of Control**: you write plain classes and a container creates and wires them. **Spring Boot** (2014) adds **auto-configuration**, **starters** and an embedded server, so an app runs from one `main()`. This set targets **Boot 4.1 / Framework 7.0** on Java 17+.

NOTES

- **Convention over configuration**: defaults back off when you define your own
- Framework = container, MVC, data access, AOP, test support
- Boot = opinionated setup for Framework + other Spring projects
- Packages are `jakarta.*`, not `javax.*`, since Boot 3
- New Boot minor release every 6 months (May, November)

HELLO, WORLD — A COMPLETE APP

```java
@SpringBootApplication            // config + auto-config + scanning
@RestController                   // methods return response bodies
public class DemoApplication {

    @GetMapping("/")              // GET http://localhost:8080/
    String hello() {
        return "Hello, World";    // -> Hello, World
    }

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

CODE EXPLAINED

- → `@SpringBootApplication` makes this class the configuration root
- → `spring-boot-starter-webmvc` on the classpath → embedded **Tomcat**
- → `@RestController` + `@GetMapping` map a request to a method
- → `SpringApplication.run()` builds the `ApplicationContext`
- → No XML, no `web.xml`, no external server to install

RUN IT

```
./mvnw spring-boot:run      # or: ./gradlew bootRun
```

SPRING PROJECTS

PROJECT	GIVES YOU
Framework	Container, MVC, AOP
Boot	Auto-config, starters
Data	Repositories (JPA…)
Security	Authn and authz
Cloud / AI	Microservices, LLMs

CORE VOCABULARY

TERM	MEANING
Bean	Object the container manages
IoC	Container builds objects
DI	Dependencies passed in
Starter	Dependency bundle
Auto-config	Beans added by Boot

VERSIONS

BOOT	FRAMEWORK · NOTES
4.1 (2026)	7.0 · Java 17–25
4.0 (Nov 2025)	7.0 · Jakarta EE 11
3.5 (2025)	6.2 · last 3.x
3.0 (2022)	6.0 · `jakarta.*`
2.7	5.3 · EOL

⚠ COMMON MISTAKES

- Treating Spring and Spring Boot as one thing; Boot configures Spring
- Copying old tutorials that import `javax.persistence`; Boot 3+ needs `jakarta`
- Calling `new MyService()`; the container never sees it, so nothing is injected

✓ TIPS & BEST PRACTICES

- Generate every project at **start.spring.io**
- Check the Boot version first when reading answers online; 2.x, 3.x and 4.x differ
- Run once with `--debug` to see which auto-configs applied

QUICK SUMMARY

- `Spring Framework`: IoC container + modules
- `Spring Boot`: auto-config + starters
- `@SpringBootApplication`: app root
- `SpringApplication.run()`: starts it

Creating a Project

GENERATING A PROJECT

Start at **start.spring.io** (Spring Initializr), your IDE's new-project wizard, or `curl`. Pick **Maven** or **Gradle**, the Boot and Java versions and dependencies, and you get a working build, a **wrapper** script, a main class and a test. Versions come from the Boot **BOM**, so you rarely write one yourself.

NOTES

- The wrapper (`./mvnw` , `./gradlew`) needs no global Maven/Gradle
- Maven inherits from `spring-boot-starter-parent`
- Gradle applies the `org.springframework.boot` plugin
- Java 17 minimum; 21 or 25 (LTS) recommended

MAVEN: POM.XML (TRIMMED)

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>         <!-- the only version you set -->
</parent>
<properties><java.version>21</java.version></properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
</dependencies>
```

GRADLE: BUILD.GRADLE.KTS

```
plugins {
    java
    id("org.springframework.boot") version "4.1.0"
    id("io.spring.dependency-management") version "1.1.7"
}
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-webmvc")
}
```

CODE EXPLAINED

- → The parent sets plugin config, encoding and the version **BOM**
- → Starters are listed **without** a version; the BOM supplies it
- → Gradle **toolchain** picks the JDK used to compile
- → The Boot plugin adds `bootRun` , `bootJar` , `bootBuildImage`
- → No parent allowed? Import `spring-boot-dependencies` as a BOM

INITIALIZR FIELDS

FIELD	TYPICAL VALUE
Project	Maven / Gradle-Kotlin
Spring Boot	Latest GA release
Group	com.example
Artifact	demo (jar name)
Java	21 or 25

FROM THE TERMINAL

FLAG	MEANING
starter.zip	curl endpoint
-d dependencies=	webmvc,data-jpa
-d type=	gradle-project-kotlin
-d javaVersion=	21
-o demo.zip	Output file

WHAT YOU GET

FILE	PURPOSE
pom.xml	Build definition
mvnw / gradlew	Build wrappers
*Application.java	Main class
application.properties	Config
*ApplicationTests	Context test

⚠ COMMON MISTAKES

- Adding `<version>` to a starter; it overrides the BOM and mixes versions
- Choosing a SNAPSHOT or milestone Boot version for real work
- Using a global `mvn` instead of `./mvnw` , so builds differ per machine

✓ TIPS & BEST PRACTICES

- Commit the wrapper files so CI uses the same build-tool version
- Use Initializr's **Explore** button to preview the build file
- Override one managed version with a property, e.g. `<postgresql.version>`

QUICK SUMMARY

- `start.spring.io` : new project
- `starter-parent` : BOM + plugins
- Boot Gradle plugin: `bootRun` , `bootJar`
- `./mvnw` / `./gradlew` : use the wrapper

Starters & Dependency Management

STARTERS

A **starter** pulls in everything one feature needs: the library, Spring's integration and Boot's auto-configuration. `spring-boot-starter-data-jpa` brings Hibernate, Spring Data JPA, a pool and transactions. **Boot 4** split auto-configuration into modules, so starters are more focused and each has a **`-test`** partner.

NOTES

- Official: `spring-boot-starter-*` ; third-party: `*-spring-boot-starter`
- Boot 4 renamed `starter-web` → `starter-webmvc`
- Flyway now needs `spring-boot-starter-flyway`
- `spring-boot-starter-classic` restores Boot 3-style all-in-one
- See the tree: `./mvnw dependency:tree`

COMMON DEPENDENCIES (MAVEN)

```
<dependency>                          <!-- REST + Tomcat -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
  <exclusions>                        <!-- drop Tomcat... -->
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>                          <!-- ...use Jetty -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

GRADLE

```
implementation("org.springframework.boot:spring-boot-starter-data-jpa")
runtimeOnly("org.postgresql:postgresql")
testImplementation("org.springframework.boot:spring-boot-starter-webmvc-test")
```

CODE EXPLAINED

- → Versions come from the BOM; only non-managed libs need one
- → `<exclusions>` removes a transitive dependency (here Tomcat)
- → Undertow was removed in Boot 4; Jetty is the alternative
- → `runtimeOnly` : needed to run, not compile (JDBC drivers)
- → `-test` starters bring that technology's test slices

WEB & DATA

STARTER	BRINGS
webmvc	MVC, Tomcat, Jackson
webflux	Reactive web, Netty
validation	Bean Validation
data-jpa	Hibernate, repositories
jdbc	JdbcClient, HikariCP

SECURITY & OPS

STARTER	BRINGS
security	Spring Security
actuator	Health, metrics
cache	Cache abstraction
thymeleaf	HTML templates
docker-compose	Dev services

MAVEN ↔ GRADLE

MAVEN	GRADLE
compile	implementation
runtime	runtimeOnly
provided	compileOnly
test	testImplementation
optional	developmentOnly

⚠ COMMON MISTAKES

- Using `starter-web` in new Boot 4 builds; it still works but is deprecated
- Adding `flyway-core` alone; Boot 4 won't auto-configure it without the starter
- Expecting `@WebMvcTest` without `spring-boot-starter-webmvc-test`

✓ TIPS & BEST PRACTICES

- Keep the starter list small; each adds auto-config and startup time
- Upgrading from 3.x? Start with `starter-classic` , then slim down
- Use `dependency:tree -Dincludes=groupId` to find version clashes

QUICK SUMMARY

- `spring-boot-starter-*` : one feature
- BOM: manages versions
- `-test` starters: test slices
- `<exclusions>` : swap transitive deps

Application Structure

HOW A BOOT APP IS LAID OUT

One class carries **`@SpringBootApplication`** and its `main()` calls **`SpringApplication.run()`**. The annotation combines `@SpringBootConfiguration`, `@EnableAutoConfiguration` and `@ComponentScan`. Scanning starts at the main class's package and goes down, so the main class lives in the **root package**.

NOTES

- Code in `src/main/java`, config in `src/main/resources`
- A class outside the root package is **not scanned**
- Package by **feature** (`order/`, `customer/`), not by layer
- Tests mirror the same packages under `src/test/java`

MAIN CLASS & CUSTOMIZATION

```java
package com.example.shop;                    // root package

@SpringBootApplication
public class ShopApplication {
    public static void main(String[] args) {
        var app = new SpringApplication(ShopApplication.class);
        app.setBannerMode(Banner.Mode.OFF);  // optional tweaks
        app.setAdditionalProfiles("local");
        app.run(args);
    }
}
```

CODE EXPLAINED

→ One line usually suffices: `SpringApplication.run(Shop.class, args)`

→ `setAdditionalProfiles()` adds to `spring.profiles.active`

→ `order/`, `customer/`, `config/` are sub-packages → scanned

→ `run()` returns a `ConfigurableApplicationContext`

PACKAGE-BY-FEATURE LAYOUT

```
com/example/shop/
├── ShopApplication.java       # @SpringBootApplication
├── order/     OrderController  OrderService  Order
├── customer/  CustomerController  CustomerService
└── config/    SecurityConfig  WebConfig
```

@SPRINGBOOTAPPLICATION

ATTRIBUTE	USE
`exclude`	Turn off an auto-config
`scanBasePackages`	Scan more packages
`scanBasePackageClasses`	Type-safe version
`proxyBeanMethods`	Lite @Bean mode

SRC/MAIN/RESOURCES

PATH	HOLDS
`application.yml`	Configuration
`static/`	CSS, JS, images
`templates/`	Thymeleaf views
`db/migration/`	Flyway scripts
`banner.txt`	Startup banner

RUN AT STARTUP

HOOK	RECEIVES
`CommandLineRunner`	`String... args`
`ApplicationRunner`	Parsed arguments
`ApplicationReadyEvent`	App fully started
`@Order(n)`	Order of runners

⚠ COMMON MISTAKES

- Main class in a sub-package (`shop.app`); sibling packages aren't scanned
- Main class in the default package; Boot scans every jar on the classpath
- Adding `@ComponentScan` with other packages, which replaces the default

✓ TIPS & BEST PRACTICES

- Keep `main()` to one line; put setup in `@Configuration` so tests get it
- Prefer `scanBasePackageClasses` over strings so refactors don't break it
- Make feature-internal classes package-private

QUICK SUMMARY

- `@SpringBootApplication`: config + scan
- Main class: root package
- `resources/`: config, static, templates
- `CommandLineRunner`: startup code

Running, Building & DevTools

RUN, PACKAGE, REPEAT

In development run with **`spring-boot:run`** or **`bootRun`**. For deployment, `package` / `bootJar` builds an **executable fat jar** with your code, all dependencies and the embedded server; run it with `java -jar`. **DevTools** restarts the app when compiled classes change.

NOTES

- DevTools restarts fast because it reloads only **your** classes
- DevTools turns itself off under `java -jar`
- Templates and static files refresh without a restart
- `docker-compose` starter starts `compose.yaml` services

RUN & BUILD

```
./mvnw spring-boot:run                        # dev run
./mvnw spring-boot:run -Dspring-boot.run.profiles=local
./mvnw clean package                          # target/demo-0.0.1.jar
./mvnw package -DskipTests

./gradlew bootRun --args='--server.port=9090'
./gradlew bootJar                             # build/libs/demo-0.0.1.jar

java -jar demo.jar --spring.profiles.active=prod
SERVER_PORT=8081 java -jar demo.jar           # env var override
```

DEVTOOLS (MAVEN)

```
<artifactId>spring-boot-devtools</artifactId>
<scope>runtime</scope> <optional>true</optional>
```

CODE EXPLAINED

- → `--name=value` arguments override every other property source
- → `-Dspring-boot.run.profiles` is a Maven plugin flag
- → `package` runs tests first; avoid `-DskipTests` in CI
- → Gradle: `developmentOnly("…:spring-boot-devtools")`
- → `optional` keeps DevTools out of the packaged jar

MAVEN VS GRADLE

GOAL	MAVEN / GRADLE
Run	`spring-boot:run` / `bootRun`
Fat jar	`package` / `bootJar`
Image	`build-image` / `bootBuildImage`
Native	`native:compile` / `nativeCompile`

SPRING.DEVTOOLS.*

PROPERTY	EFFECT
`restart.enabled`	Toggle restarts
`restart.exclude`	Paths to ignore
`restart.additional-paths`	Watch more paths
`livereload.enabled`	Browser refresh

TRIGGER A RESTART

IDE	ACTION
IntelliJ	Build Project (⌘F9)
IntelliJ auto	Build automatically
VS Code	Save the file
Eclipse	Save (auto build)

⚠ COMMON MISTAKES

- Expecting DevTools to restart on edit; the IDE must **compile** first
- Shipping DevTools by leaving out `optional` / `developmentOnly`
- Running Gradle's `-plain.jar`; it has no dependencies inside

✓ TIPS & BEST PRACTICES

- Add `spring-boot-docker-compose` and Postgres starts with the app
- Pass config at deploy time with env vars like `SERVER_PORT`
- Set the jar name with `<finalName>` or `archiveFileName`

QUICK SUMMARY

- `bootRun` / `spring-boot:run`: dev
- `bootJar` / `package`: fat jar
- `java -jar app.jar --k=v`: run
- DevTools: restart on compile

The IoC Container

THE CONTAINER

The **IoC container** creates your objects (**beans**), injects their dependencies and manages their lifecycle. In Boot it is an **`ApplicationContext`**, which adds events, messages and environment access to the basic `BeanFactory`. At startup it registers **bean definitions**, then creates every singleton in dependency order.

NOTES

- **Bean definition** = recipe; **bean** = the instance
- Singletons are created **at startup**, so wiring errors fail fast
- Default name: class name, first letter lower-case
- Prefer injection over `getBean()` (Sheet 08)

LOOKING INTO THE CONTEXT

```
public static void main(String[] args) {
    var ctx = SpringApplication.run(App.class, args);

    var svc  = ctx.getBean(OrderService.class);          // by type
    var same = ctx.getBean("orderService", OrderService.class);
    System.out.println(svc == same);                     // true

    ctx.getBeanDefinitionCount();        // e.g. 142
    ctx.getBeansOfType(Clock.class);     // Map<String, Clock>
    ctx.containsBean("dataSource");      // true
}

@Bean
CommandLineRunner seed(OrderRepository repo) {  // runs after startup
    return args -> repo.save(new Order("first"));
}
```

CODE EXPLAINED

→ `run()` returns the started `ConfigurableApplicationContext`

→ `getBean(Type)` throws if zero or several beans match

→ `svc == same` → singleton scope (Sheet 11)

→ `getBeansOfType()` returns name → bean, empty if none

→ The runner's `repo` parameter is injected like a constructor arg

CONTEXT METHODS

METHOD	RETURNS
`getBean(Class)`	The one bean
`getBeansOfType()`	Map of matches
`getBeanProvider()`	Lazy lookup
`getEnvironment()`	Properties
`publishEvent()`	Fires an event

STARTUP ORDER

STEP	WHAT HAPPENS
1	Environment loaded
2	Definitions registered
3	Singletons created
4	Web server started
5	Runners, ready event

CONTAINER EXTRAS

FEATURE	API
Events	`@EventListener`
Messages	`MessageSource`
Resources	`ResourceLoader`
Shutdown	`ctx.close()`

⚠ COMMON MISTAKES

- Calling `getBean()` all over (service locator); it hides dependencies
- Creating a second context with `new AnnotationConfigApplicationContext()`
- Heavy work in constructors; it slows startup and every test

✓ TIPS & BEST PRACTICES

- List every bean with Actuator's `/actuator/beans` (Sheet 57)
- `spring.main.lazy-initialization=true` speeds dev startup
- Use `ApplicationReadyEvent` for work that needs the server up

QUICK SUMMARY

- `ApplicationContext`: the container
- Bean: object it manages
- `getBean()`: lookup, rarely needed
- `CommandLineRunner`: after startup

Stereotypes & Component Scanning

STEREOTYPES

Mark a class with a **stereotype** and component scanning registers it as a bean. **`@Component`** is generic; **`@Service`**, **`@Repository`** and **`@Controller`** name the layer. `@Repository` also translates persistence exceptions into Spring's `DataAccessException`, and `@RestController` adds `@ResponseBody` to every method.

NOTES

- `@Configuration` is a `@Component` too, so it is scanned
- Spring Data repository **interfaces** need no annotation
- Override the name: `@Service("billing")`
- Custom stereotype: meta-annotate with `@Component`

ONE CLASS PER LAYER

```
@RestController
@RequestMapping("/orders")
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) { this.service = service; }

    @GetMapping("/{id}")
    Order get(@PathVariable long id) { return service.find(id); }
}

@Service
class OrderService {
    private final OrderRepository repo;
    OrderService(OrderRepository repo) { this.repo = repo; }
    Order find(long id) { return repo.findById(id).orElseThrow(); }
}

interface OrderRepository extends JpaRepository<Order, Long> { }
```

CODE EXPLAINED

→ All three are found because they sit under the main class's package

→ A single constructor is injected automatically

→ Classes may be package-private; Spring doesn't need `public`

→ Spring Data generates the repository implementation

STEREOTYPES

ANNOTATION	MEANING
`@Component`	Generic bean
`@Service`	Business logic
`@Repository`	Data access + translation
`@Controller`	MVC, returns views
`@RestController`	Returns bodies

@COMPONENTSCAN

ATTRIBUTE	EFFECT
`basePackages`	Packages to scan
`basePackageClasses`	Type-safe markers
`includeFilters`	Add unannotated types
`excludeFilters`	Skip matches
`lazyInit`	Create on first use

FILTERTYPE

TYPE	MATCHES BY
`ANNOTATION`	Annotation present
`ASSIGNABLE_TYPE`	Class or subtype
`REGEX`	Class name regex
`CUSTOM`	Your `TypeFilter`

⚠ COMMON MISTAKES

- Putting `@Service` on an **interface**; annotate the implementation class
- Two classes with the same simple name → `ConflictingBeanDefinitionException`
- Annotating a class that a `@Bean` method also creates → two beans

✓ TIPS & BEST PRACTICES

- Use `@Service` / `@Repository` over `@Component`; they document intent
- Keep controllers thin so logic is testable without HTTP
- Use `@Bean` for library classes you can't annotate

QUICK SUMMARY

- `@Component`: generic bean
- `@Service` / `@Repository`: layers
- `@RestController`: JSON controller
- Scan root: main class's package

Dependency Injection

INJECTING DEPENDENCIES

With **dependency injection** a bean receives its collaborators instead of creating them. **Constructor injection** is preferred: fields are `final`, required and visible, and tests can call `new`. With a **single constructor** Spring uses it automatically, so `@Autowired` is optional.

NOTES

- Matching is **by type**, then by name if several match
- Missing bean → `NoSuchBeanDefinitionException` at startup
- `List<T>` / `Map<String,T>` inject **every** bean of a type
- Circular dependencies fail at startup by default

CONSTRUCTOR INJECTION (PREFERRED)

```
@Service
class CheckoutService {
    private final PaymentGateway payments;        // required
    private final List<DiscountRule> rules;       // all rule beans
    private final ObjectProvider<AuditLog> audit; // optional

    CheckoutService(PaymentGateway payments, List<DiscountRule> rules,
                    ObjectProvider<AuditLog> audit) {
        this.payments = payments;
        this.rules = rules;
        this.audit = audit;
    }

    void checkout(Cart cart) {
        rules.forEach(r -> r.apply(cart));
        payments.charge(cart.total());
        audit.ifAvailable(a -> a.record(cart));  // only if bean exists
    }
}
```

CODE EXPLAINED

- → `final` fields are set once, so the object is always complete
- → `List<DiscountRule>` follows `@Order` on the beans
- → `ObjectProvider` defers lookup and tolerates a missing bean
- → Unit test: `new CheckoutService(fake, List.of(), provider)`

FIELD & SETTER INJECTION (AVOID)

```
@Autowired private PaymentGateway payments;        // field
@Autowired void setAudit(AuditLog a) { audit = a; } // setter
```

INJECTION STYLES

STYLE	USE FOR
`Constructor`	Default choice
`Setter`	Optional, changeable
`Field`	Tests, demos only
`@Bean param`	Config classes

PARAMETER TYPES

TYPE	GETS
`T`	Exactly one bean
`Optional<T>`	Bean or empty
`@Nullable T`	Bean or `null`
`List<T>`	All beans, ordered
`Map<String,T>`	Name → bean

OBJECTPROVIDER

METHOD	BEHAVIOR
`getObject()`	Bean or exception
`getIfAvailable()`	Bean or `null`
`ifAvailable(fn)`	Run if present
`getIfUnique()`	Null if 0 or many
`orderedStream()`	All beans

⚠ COMMON MISTAKES

- Private field injection: the class can't be unit-tested without Spring
- Hiding a circular dependency with `@Lazy` instead of splitting the logic
- Adding a 2nd constructor without `@Autowired` on one; Spring can't choose

✓ TIPS & BEST PRACTICES

- Over ~5 constructor parameters? The class is doing too much
- Inject interfaces so tests can pass fakes
- Lombok `@RequiredArgsConstructor` writes the constructor

QUICK SUMMARY

- Constructor injection: `final` fields
- `ObjectProvider<T>`: optional/lazy
- `List<T>`: every bean of a type
- Field injection: tests only

@Configuration & @Bean

JAVA-BASED CONFIGURATION

A **`@Configuration`** class holds **`@Bean`** methods: the return value becomes a bean and the parameters are injected. Use `@Bean` for classes you can't annotate (`Clock`, `RestClient`) or that need custom setup. In the default mode, calling one `@Bean` method from another returns the **same singleton**.

NOTES

- Bean name = method name; override with `@Bean("name")`
- `proxyBeanMethods = false` skips the proxy ("lite mode")
- `@Import(Other.class)` adds configs that aren't scanned
- A public `close()` / `shutdown()` is called on exit

DEFINING BEANS IN CODE

```
@Configuration(proxyBeanMethods = false)
@Import(CacheConfig.class)                    // pull in another config
class AppConfig {

    @Bean
    Clock clock() {                           // bean "clock"
        return Clock.systemUTC();
    }

    @Bean
    RestClient github(RestClient.Builder b) {  // Boot's builder injected
        return b.baseUrl("https://api.github.com").build();
    }

    @Bean(initMethod = "start", destroyMethod = "stop")
    MessagePump pump(Clock clock) {           // uses the bean above
        return new MessagePump(clock, Duration.ofSeconds(5));
    }
}
```

CODE EXPLAINED

- → Lite mode: get dependencies via **parameters**, never by calling `clock()`
- → `RestClient.Builder` is pre-configured by Boot (Sheet 30)
- → `initMethod` / `destroyMethod` add lifecycle calls to plain classes
- → Your `@Bean` of a type Boot also defines **replaces** Boot's

@BEAN ATTRIBUTES

ATTRIBUTE	PURPOSE
`name`	Bean name, aliases
`initMethod`	Call after injection
`destroyMethod`	Call on shutdown
`autowireCandidate`	`false` = skip by type
`defaultCandidate`	Qualifier-only (6.2)

USED WITH @BEAN

ANNOTATION	EFFECT
`@Primary`	Wins ties
`@Qualifier`	Select by tag
`@Scope`	Prototype, request
`@Profile`	Some profiles only
`@Conditional…`	Only if condition

CALLING BEAN() TWICE

MODE	RESULT
`@Configuration`	Same singleton
`proxyBeanMethods=false`	New object
`@Bean in @Component`	New object

⚠ COMMON MISTAKES

- Calling a `@Bean` method in lite mode and getting a **second** instance
- Defining your own `JsonMapper` bean without realizing it replaces Boot's
- Non-`static` `@Bean` for a `BeanFactoryPostProcessor`; it inits too early

✓ TIPS & BEST PRACTICES

- Use lite mode + parameter injection; Boot's own configs do
- Prefer Boot's `*Customizer` beans to replacing a whole bean
- One config class per feature (`SecurityConfig`, `HttpConfig`)

QUICK SUMMARY

- `@Configuration`: bean definitions
- `@Bean`: return value is a bean
- Parameters: injected dependencies
- `proxyBeanMethods=false`: lite mode

Resolving Ambiguity

WHEN SEVERAL BEANS MATCH

Injection is by **type**. When two beans share a type, Spring needs a tie-breaker or startup fails with `NoUniqueBeanDefinitionException`. Mark the default with **`@Primary`**, pick one explicitly with **`@Qualifier`**, demote a bean with **`@Fallback`** (6.2+), or let the **parameter name** match the bean name.

NOTES

- Resolution order: `@Qualifier` → `@Primary` → non-`@Fallback` → name
- `@Qualifier` on the bean and on the injection point must match
- Parameter-name matching needs the `-parameters` compiler flag (Boot sets it)
- `List<T>` injection sidesteps the problem by taking all of them

THREE WAYS TO CHOOSE

```
interface Notifier { void send(String msg); }

@Component @Primary                          // default choice
class EmailNotifier implements Notifier { /* ... */ }

@Component @Qualifier("sms")                 // tagged
class SmsNotifier implements Notifier { /* ... */ }

@Component @Fallback                         // used only if nothing else
class LogNotifier implements Notifier { /* ... */ }

@Service
class AlertService {
    AlertService(Notifier main,                        // EmailNotifier
                 @Qualifier("sms") Notifier urgent,    // SmsNotifier
                 Notifier logNotifier) {               // by name
    }
}
```

CODE EXPLAINED

→ `main` gets the `@Primary` bean

→ `@Qualifier("sms")` matches the bean tagged `sms` (or named `sms`)

→ `logNotifier` matches the bean **name** of `LogNotifier`

→ A custom qualifier is type-safe; no typo-prone strings

CUSTOM QUALIFIER ANNOTATION

```
@Qualifier @Retention(RUNTIME) @Target({TYPE, PARAMETER, METHOD})
@interface Urgent { }        // then: @Urgent Notifier n
```

TIE-BREAKERS

ANNOTATION	EFFECT
`@Primary`	Default when ambiguous
`@Qualifier("x")`	Select by tag/name
`@Fallback`	Lowest priority (6.2)
`@Priority(n)`	Lower n wins
`@Order(n)`	Order in `List<T>` only

ERRORS YOU'LL SEE

EXCEPTION	CAUSE
`NoUniqueBean…`	2+ beans, no winner
`NoSuchBean…`	No bean of that type
`UnsatisfiedDep…`	Wrapper for the above
`BeanCurrentlyIn…`	Circular dependency

@QUALIFIER PLACEMENT

ON	MEANS
`Class`	Tag this bean
`@Bean method`	Tag that bean
`Constructor param`	Want that tag
`Field`	Want that tag

⚠ COMMON MISTAKES

- Two `@Primary` beans of one type; you're back to ambiguity
- Using `@Order` to pick a single bean; it only sorts collections
- Relying on parameter names, then renaming a bean in a refactor

✓ TIPS & BEST PRACTICES

- Prefer one `@Primary` + `@Qualifier` for the exceptions
- Use custom qualifier annotations in larger codebases
- Use `@Fallback` for library defaults that apps may override

QUICK SUMMARY

- `@Primary`: default bean
- `@Qualifier`: pick explicitly
- `@Fallback`: only if nothing else
- Param name = bean name: last resort

Bean Scopes

HOW LONG A BEAN LIVES

A bean's **scope** decides how many instances exist. The default, **singleton**, means one instance per container, shared by everyone, so it must be **stateless or thread-safe**. **Prototype** creates a new instance every time it's requested. Web apps add **request**, **session** and **application** scopes, tied to the HTTP lifecycle.

NOTES

- Singletons are created at startup; prototypes on demand
- Spring does **not** call destroy methods on prototypes
- A prototype injected into a singleton is created **once**
- Use a **scoped proxy** or `ObjectProvider` to get fresh instances

DECLARING SCOPES

```
@Component                                      // singleton (default)
class PriceCalculator { }

@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE) // new each lookup
class ReportBuilder { }

@Component
@RequestScope                                   // one per HTTP request
class RequestContext { String traceId; }

@Service
class ReportService {
    private final ObjectProvider<ReportBuilder> builders;
    private final RequestContext ctx;             // proxy → current request

    ReportService(ObjectProvider<ReportBuilder> builders, RequestContext ctx) {
        this.builders = builders;
        this.ctx = ctx;
    }
    Report build() { return builders.getObject().with(ctx.traceId).build(); }
}
```

CODE EXPLAINED

→ `builders.getObject()` returns a **new** `ReportBuilder` each call

→ `@RequestScope` creates a proxy, so a singleton can hold it

→ The proxy forwards to the bean of the **current** request

→ Using request scope outside a request throws `IllegalStateException`

BUILT-IN SCOPES

SCOPE	INSTANCES
`singleton`	One per container
`prototype`	New per lookup
`request`	One per HTTP request
`session`	One per HTTP session
`application`	One per ServletContext

SHORTCUT ANNOTATIONS

ANNOTATION	EQUALS
`@RequestScope`	request + proxy
`@SessionScope`	session + proxy
`@ApplicationScope`	application + proxy
`@Lazy`	Create on first use

PROXYMODE

VALUE	PROXY TYPE
`DEFAULT / NO`	None
`TARGET_CLASS`	CGLIB subclass
`INTERFACES`	JDK interface proxy

⚠ COMMON MISTAKES

- Keeping per-user data in a field of a singleton; all users share it
- Expecting a prototype field in a singleton to change on each call
- Using `@SessionScope` beans in stateless JWT APIs where no session exists

✓ TIPS & BEST PRACTICES

- Default to stateless singletons; pass state as method arguments
- Use `ObjectProvider<T>` to get a fresh prototype on demand
- `@Lazy` on an injection point delays creation of expensive beans

QUICK SUMMARY

- `singleton` : one shared instance
- `prototype` : new per lookup
- `@RequestScope` : per request, proxied
- `ObjectProvider` : fresh instances

Bean Lifecycle & Callbacks

LIFECYCLE OF A BEAN

After Spring creates a bean and injects its dependencies, it runs **initialization callbacks**; at shutdown it runs **destruction callbacks**. The usual choice is **`@PostConstruct`** and **`@PreDestroy`** (from `jakarta.annotation`). **`BeanPostProcessor`** hooks run around every bean's init and are how Spring adds proxies, while **`SmartLifecycle`** starts and stops background work in phases.

NOTES

- Constructor runs **before** setter/field injection; use `@PostConstruct` for setup that needs them
- Destroy callbacks run on `ctx.close()` or normal JVM shutdown
- Prototype beans get init callbacks but **no** destroy callbacks
- Graceful shutdown (default) lets in-flight requests finish first

INIT & DESTROY CALLBACKS

```
@Component
class FeedPoller implements SmartLifecycle {
    private final FeedClient client;
    private volatile boolean running;

    FeedPoller(FeedClient client) { this.client = client; } // 1. inject

    @PostConstruct void init() { client.connect(); }    // 2. after inject
    @PreDestroy void close()   { client.disconnect(); } // last

    @Override public void start() { running = true; }   // all beans ready
    @Override public void stop()  { running = false; }  // before @PreDestroy
    @Override public boolean isRunning() { return running; }
}

@Component
class TimingPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessAfterInitialization(Object bean, String name) {
        return bean;           // could wrap it in a proxy here
    }
}
```

CODE EXPLAINED

- → `SmartLifecycle.start()` runs after **all** singletons are ready
- → `stop()` runs before destroy callbacks, in reverse phase order
- → `@PostConstruct` method: no args, return `void`, any visibility
- → `BeanPostProcessor` sees every bean the context creates

CALLBACK ORDER

STEP	CALLBACK
1	Constructor + injection
2	*Aware interfaces
3	@PostConstruct
4	afterPropertiesSet()
5	@Bean(initMethod)

DESTROY ORDER

STEP	CALLBACK
1	SmartLifecycle.stop()
2	@PreDestroy
3	DisposableBean.destroy()
4	@Bean(destroyMethod)

AWARE INTERFACES

INTERFACE	INJECTS
BeanNameAware	Own bean name
ApplicationContextAware	The context
EnvironmentAware	Environment
ResourceLoaderAware	Resource loader

⚠ COMMON MISTAKES

- Using an `@Autowired` field inside the constructor; it's still `null` there
- Implementing `InitializingBean` in app code; it couples you to Spring
- Long-running work in `@PostConstruct`; it blocks startup

✓ TIPS & BEST PRACTICES

- Prefer constructor injection, then the constructor can do the setup
- Use `SmartLifecycle` (or `ApplicationReadyEvent`) for background threads
- Set `spring.lifecycle.timeout-per-shutdown-phase` for slow shutdowns

QUICK SUMMARY

- `@PostConstruct`: after injection
- `@PreDestroy`: before shutdown
- `BeanPostProcessor`: wrap any bean
- `SmartLifecycle`: start/stop work

Sheet 13

application.properties & YAML

application.yml · property syntax · relaxed binding · placeholders ${} · spring.config.import · .env-style overrides

WHERE CONFIGURATION LIVES

Boot reads settings from **`application.properties`** or **`application.yml`** in `src/main/resources`, then layers environment variables, command-line arguments and other sources on top. Keys use **relaxed binding**, so `server.port`, SERVER_PORT and `server-port` all reach the same property. Values can reference others with **`${...}`** placeholders.

NOTES

- Use **one** format per app; if both exist, `.properties` wins
- YAML: indentation matters, two spaces, no tabs
- `---` splits one YAML file into several documents
- `spring.config.import` pulls in more files, `.env` files or config servers

APPLICATION.YML

```yaml
spring:
  application:
    name: shop
  datasource:
    url: jdbc:postgresql://localhost:5432/shop
    username: ${DB_USER:shop}            # env var with default
    password: ${DB_PASSWORD}
  config:
    import: optional:file:.env[.properties]

server:
  port: 8080
  servlet:
    context-path: /api

app:
  base-url: https://shop.example.com
  checkout-url: ${app.base-url}/checkout  # reuse another key
```

CODE EXPLAINED

- → `${DB_USER:shop}` uses env var `DB_USER`, or `shop` if missing
- → `optional:` means no error when the file doesn't exist
- → `file:.env[.properties]` reads a `.env` file as properties
- → `app.*` is your own namespace; bind it with Sheet 15

SAME IN .PROPERTIES

```properties
server.port=8080
spring.datasource.username=${DB_USER:shop}
```

PRECEDENCE (HIGH → LOW)

SOURCE	EXAMPLE
Command line	`--server.port=9000`
Env variables	`SERVER_PORT=9000`
Profile files	`application-prod.yml`
application.yml	In the jar
Defaults	Set in code

RELAXED BINDING

FORM	WHERE
`my.main-project`	Files (preferred)
`my.mainProject`	Camel case
`MY_MAINPROJECT`	Env variables
`MY_LIST_0_NAME`	Env list index

COMMON KEYS

KEY	SETS
`server.port`	HTTP port
`server.servlet.context-path`	URL prefix
`logging.level.*`	Log levels
`spring.application.name`	App name

⚠ COMMON MISTAKES

- Committing real passwords to `application.yml`; use env vars or a vault
- Tabs in YAML; Boot fails with a parse error
- Unquoted YAML values like `on` / `no` / `08`, parsed as booleans or octal

✓ TIPS & BEST PRACTICES

- Run with `--debug` or check `/actuator/env` to see where a value came from
- Put local-only overrides in a git-ignored `application-local.yml`
- Add `spring-boot-configuration-processor` for IDE autocompletion of your keys

QUICK SUMMARY

- `application.yml`: main config file
- `${KEY:default}`: placeholder
- Env vars / CLI override files
- Relaxed binding: `SERVER_PORT`

@Value & Environment

READING SINGLE VALUES

`@Value` injects one property into a field or constructor parameter, with an optional default after a colon. It also evaluates **SpEL** expressions in **`#{...}`**. For lookups at runtime, inject the **`Environment`**. For more than two or three related values, switch to `@ConfigurationProperties` (Sheet 15).

NOTES

- `${...}` = property placeholder; `#{...}` = SpEL expression
- Missing key with no default → startup fails
- `@Value` works on constructor parameters too (best for testing)
- `@PropertySource` adds `.properties` files (not YAML)

@VALUE AND ENVIRONMENT

```
@Service
class MailService {
    private final String from;
    private final int retries;
    private final List<String> admins;
    private final Environment env;

    MailService(@Value("${mail.from}") String from,
                @Value("${mail.retries:3}") int retries,          // default 3
                @Value("${mail.admins}") List<String> admins,     // a,b,c
                Environment env) {
        this.from = from; this.retries = retries;
        this.admins = admins; this.env = env;
    }

    boolean sandbox() {
        return env.getProperty("mail.sandbox", Boolean.class, false);
    }
    boolean isProd() { return env.matchesProfiles("prod"); }
}

@Value("#{T(java.lang.Math).random() * 100}") double roll;  // SpEL
@Value("${app.timeout:PT30S}") Duration timeout;            // converted
```

CODE EXPLAINED

- → `${mail.retries:3}` → `3` when the key is absent
- → Comma-separated values convert to `List` or arrays
- → `env.getProperty(key, type, default)` reads and converts at runtime
- → `Duration`, `DataSize`, enums and `Path` convert automatically

@VALUE SYNTAX

EXPRESSION	RESULT
`${key}`	Property value
`${key:dflt}`	With default
`${key:}`	Empty string default
`#{2 * 3}`	SpEL → 6
`#{@bean.name}`	Another bean's value

ENVIRONMENT METHODS

METHOD	RETURNS
`getProperty(k)`	`String` or null
`getProperty(k,T,d)`	Converted + default
`getRequiredProperty`	Value or exception
`containsProperty`	`boolean`
`matchesProfiles(…)`	Profile check

AUTO CONVERSION

TEXT	BECOMES
`30s / PT30S`	`Duration`
`10MB`	`DataSize`
`a,b,c`	`List<String>`
`true`	`boolean`

⚠ COMMON MISTAKES

- `@Value` on a `static` field; Spring never sets it
- Writing `@Value("mail.from")` without `${}`; you get the literal text
- Using `@Value` in a class created with `new`; nothing is injected

✓ TIPS & BEST PRACTICES

- Group related keys into `@ConfigurationProperties` instead of many `@Value` s
- Give defaults for optional keys so local runs need no config
- Use `Environment` only when the key isn't known until runtime

QUICK SUMMARY

- `@Value("${k:default}")` : one value
- `#{...}` : SpEL expression
- `Environment.getProperty()` : runtime
- `@PropertySource` : extra files

@ConfigurationProperties

TYPE-SAFE CONFIGURATION

`@ConfigurationProperties` binds a whole prefix of keys onto a class or **record**, converting types and nesting objects, lists and maps. Add **`@Validated`** with Bean Validation annotations to fail startup on bad values. It's the recommended way to read your own settings; `@Value` is for one-offs.

NOTES

- Records bind through the constructor and are immutable
- Register with `@ConfigurationPropertiesScan` or `@EnableConfigurationProperties`
- Binding is relaxed: `max-size`, `maxSize` and `MAX_SIZE` all work
- `spring-boot-configuration-processor` adds IDE autocomplete

BINDING A PREFIX TO A RECORD

```
@ConfigurationProperties("app.storage")
@Validated
public record StorageProperties(
        @NotBlank String bucket,
        @DefaultValue("10MB") DataSize maxUpload,
        @DefaultValue("30s") Duration timeout,
        List<String> allowedTypes,
        Map<String, String> tags,
        Retry retry) {

    public record Retry(@DefaultValue("3") int attempts) { }
}

@SpringBootApplication
@ConfigurationPropertiesScan          // finds @ConfigurationProperties
class App { }
```

CODE EXPLAINED

- → `@DefaultValue` sets a default for a constructor-bound field
- → `@NotBlank` fails startup if `app.storage.bucket` is missing
- → Nested record `Retry` maps to `app.storage.retry.*`
- → Inject `StorageProperties` into any bean's constructor

APPLICATION.YML

```
app.storage:
  bucket: media
  max-upload: 25MB
  allowed-types: [jpg, png]
  retry.attempts: 5
```

REGISTERING

OPTION	USE WHEN
`…PropertiesScan`	Scan your classes
`@Enable…Properties`	List types by hand
`@Bean method`	Third-party class

SUPPORTED TYPES

TYPE	YAML
`List<String>`	`[a, b]` or list
`Map<String,T>`	Nested keys
`Duration`	`30s`, `5m`
`DataSize`	`10MB`
`enum`	`MODE_A`, `mode-a`

VS @VALUE

FEATURE	@CONFIGPROPS
`Relaxed binding`	Full
`Validation`	`@Validated`
`Metadata / IDE`	Yes
`SpEL`	No

⚠ COMMON MISTAKES

- Forgetting to register the class; startup says no bean of that type
- Using a mutable JavaBean without setters; fields stay `null`
- Validation annotations without `@Validated`; they're ignored

✓ TIPS & BEST PRACTICES

- Prefer **records**: immutable, no setters, constructor binding
- Prefix with your app name (`app.`, `shop.`) to avoid clashes
- Add the configuration processor so typos show up in the IDE

QUICK SUMMARY

- `@ConfigurationProperties("prefix")`
- Records: immutable binding
- `@Validated`: fail fast on bad config
- `@ConfigurationPropertiesScan`: register

Profiles

ENVIRONMENT-SPECIFIC CONFIG

A **profile** is a named set of configuration and beans, such as `dev`, `test` or `prod`. Activate profiles with **`spring.profiles.active`**; Boot then loads **`application-{profile}.yml`** on top of the base file and registers beans marked **`@Profile`** for that profile. With none active, the `default` profile applies.

NOTES

- Later active profiles override earlier ones
- Profile files always beat the base `application.yml`
- `spring.profiles.group` activates several profiles under one name
- `spring.profiles.active` can't be set inside a profile-specific file

PROFILE-SPECIFIC BEANS

```
interface PaymentGateway { void charge(Money m); }

@Component @Profile("prod")
class StripeGateway implements PaymentGateway { /* real calls */ }

@Component @Profile("!prod")              // everything except prod
class FakeGateway implements PaymentGateway { /* logs only */ }

@Configuration @Profile({"dev", "test"})  // dev OR test
class DevDataConfig {
    @Bean CommandLineRunner seed(ProductRepository r) {
        return args -> r.save(new Product("Sample"));
    }
}
```

CODE EXPLAINED

- → `!prod` matches whenever `prod` is **not** active
- → `@Profile` on a config class affects all its `@Bean` s
- → `prod` group turns on `prod-db` and `prod-mail` too
- → `on-profile` scopes one YAML document to a profile

FILES & ACTIVATION

```
# application.yml
spring.profiles.group.prod: [prod-db, prod-mail]
---
spring.config.activate.on-profile: dev   # this document = dev only
logging.level.com.example: DEBUG
```

ACTIVATE

```
java -jar app.jar --spring.profiles.active=prod
SPRING_PROFILES_ACTIVE=dev,local ./mvnw spring-boot:run
```

@PROFILE EXPRESSIONS

EXPRESSION	ACTIVE WHEN
`"prod"`	prod is active
`"!prod"`	prod not active
`{"a", "b"}`	a OR b
`"a & b"`	a AND b
`"a | (b & !c)"`	Combined

FILE NAMING

FILE	LOADED
`application.yml`	Always
`application-dev.yml`	dev active
`application-test.yml`	test active
`application-local.yml`	local (git-ignore)

PROFILE PROPERTIES

PROPERTY	PURPOSE
`spring.profiles.active`	Profiles to use
`spring.profiles.include`	Always add these
`spring.profiles.group.*`	Named groups
`spring.profiles.default`	Fallback profile

⚠ COMMON MISTAKES

- Setting `spring.profiles.active` inside `application-dev.yml` ; it's rejected
- Building one jar per environment instead of activating profiles at runtime
- Too many profiles for small differences; use env vars for single values

✓ TIPS & BEST PRACTICES

- Use `@ActiveProfiles("test")` in tests (Sheet 52)
- Keep `prod` settings in env vars or a secret store, not in the jar
- Log active profiles at startup; Boot prints them in the first lines

QUICK SUMMARY

- `@Profile("x")` : bean only in x
- `application-x.yml` : config for x
- `spring.profiles.active` : switch on
- Groups: several profiles at once

Auto-Configuration & Conditionals

HOW BOOT CONFIGURES ITSELF

Auto-configuration classes add beans when **conditions** hold: a class is on the classpath, a property is set, or no bean of that type exists yet. Because most use **`@ConditionalOnMissingBean`**, defining your own bean makes Boot's back off. Run with **`--debug`** to see a report of which configurations matched and why.

NOTES

- Auto-configs are listed in `META-INF/spring/…AutoConfiguration.imports`
- They run **after** your own configuration
- Exclude one with `@SpringBootApplication(exclude = …)`
- Boot 4 splits them into per-technology modules

WRITING YOUR OWN

```
@AutoConfiguration
@ConditionalOnClass(Stripe.class)                       // library present
@ConditionalOnProperty(prefix = "payments", name = "enabled",
                       havingValue = "true", matchIfMissing = true)
@EnableConfigurationProperties(PaymentProperties.class)
public class PaymentAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean                           // user's bean wins
    PaymentGateway paymentGateway(PaymentProperties props) {
        return new StripeGateway(props.apiKey());
    }
}
```

CODE EXPLAINED

- → `@AutoConfiguration` = `@Configuration(proxyBeanMethods=false)` + ordering
- → `matchIfMissing = true` : enabled unless set to `false`
- → Users override the gateway just by declaring their own bean
- → The report lists **Positive** and **Negative** matches with reasons

REGISTER IT

```
# src/main/resources/META-INF/spring/
#   org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.example.payments.PaymentAutoConfiguration
```

SEE WHAT APPLIED

```
java -jar app.jar --debug       # CONDITIONS EVALUATION REPORT
```

@CONDITIONALON…

SUFFIX	MATCHES IF
…Class	Class on classpath
…MissingBean	No such bean yet
…Bean	Bean exists
…Property	Property value
…WebApplication	Web app

TURNING THINGS OFF

HOW	EXAMPLE
Annotation	`exclude = X.class`
Property	`…autoconfigure.exclude`
Own bean	Define it yourself
Feature flag	e.g. `…enabled=false`

ORDERING

ATTRIBUTE	MEANING
before	Run before these
after	Run after these
@AutoConfigureOrder	Numeric order

⚠ COMMON MISTAKES

- Annotating an auto-config class `@Component` so it's scanned, which breaks ordering
- Putting `@ConditionalOnMissingBean` on a user config class; order isn't guaranteed
- Excluding `DataSourceAutoConfiguration` to "fix" a missing URL instead of setting it

✓ TIPS & BEST PRACTICES

- Read the `--debug` report before overriding anything
- Ship custom auto-config in its own module plus a `*-spring-boot-starter`
- Test conditions with `ApplicationContextRunner`

QUICK SUMMARY

- Auto-config: conditional `@Bean` s
- `@ConditionalOnMissingBean` : yours wins
- `--debug` : conditions report
- `exclude` : turn one off

AOP Concepts & Proxies

CROSS-CUTTING CONCERNS

Aspect-oriented programming moves code that cuts across many classes (logging, timing, security, transactions) into one **aspect**. Spring AOP works with **proxies**: callers get a proxy object that runs the **advice** and then calls the real bean. `@Transactional`, `@Cacheable`, `@Async` and method security are all built this way.

NOTES

- Spring AOP intercepts **public method calls on beans** only
- Default proxy type in Boot: **CGLIB** subclass (even with interfaces)
- `final` classes or methods can't be proxied by CGLIB
- Full AspectJ (compile-time weaving) can do more but is rarely needed

PROXIES AND THE SELF-INVOCATION TRAP

```
@Service
class OrderService {

    @Transactional
    public void placeOrder(Order o) { save(o); }

    public void importAll(List<Order> orders) {
        orders.forEach(this::placeOrder);    // ❌ this.placeOrder skips the proxy
    }                                        //    → no transaction!
}

@Service
class ImportService {
    private final OrderService orders;       // injected = the proxy
    ImportService(OrderService orders) { this.orders = orders; }

    public void importAll(List<Order> list) {
        list.forEach(orders::placeOrder);    // ✅ goes through the proxy
    }
}
```

CODE EXPLAINED

- → Callers hold the **proxy**; `this` inside the bean is the **target**
- → Calls on `this` bypass all advice: transactions, caching, security
- → Fix: move the method to another bean and inject it
- → The CGLIB class name gives the proxy away in stack traces

IS IT A PROXY?

```
AopUtils.isAopProxy(orderService);          // true
orderService.getClass();                    // OrderService$$SpringCGLIB$$0
```

AOP TERMS

TERM	MEANING
`Aspect`	Module of advice
`Join point`	A method execution
`Pointcut`	Which join points
`Advice`	Code to run there
`Weaving`	Linking it in (proxy)

PROXY TYPES

TYPE	WORKS BY
`CGLIB`	Subclassing the bean
`JDK dynamic`	Implementing interfaces
`AspectJ weaving`	Bytecode at build/load

BUILT ON AOP

ANNOTATION	ADVICE DOES
`@Transactional`	Begin / commit
`@Cacheable`	Return cached value
`@Async`	Run on executor
`@PreAuthorize`	Check access
`@Retryable`	Retry on failure

⚠ COMMON MISTAKES

- Calling an annotated method on `this` and wondering why it has no effect
- Putting `@Transactional` on `private` methods; proxies can't intercept them
- Making a bean class `final` (or a Kotlin class without `open`)

✓ TIPS & BEST PRACTICES

- Annotate **public** service methods called from other beans
- Split a bean when it needs to call its own advised method
- Check `AopUtils.isAopProxy()` when an annotation seems ignored

QUICK SUMMARY

- Aspect: cross-cutting module
- Proxy: wraps the bean, runs advice
- Self-invocation: bypasses the proxy
- CGLIB: Boot's default proxy

Advice & Pointcut Expressions

WRITING AN ASPECT

An **`@Aspect`** class (also a `@Component`) holds **advice** methods. Each advice names a **pointcut**: an expression that selects methods by signature (`execution(...)`), by annotation (`@annotation(...)`) or by package (`within(...)`). **`@Around`** is the most flexible: it gets a `ProceedingJoinPoint` and decides when to call `proceed()`.

NOTES

- Needs `spring-boot-starter-aspectj` (renamed from `starter-aop` in Boot 4)
- Advice on the same join point is ordered with `@Order` on the aspect
- Prefer **annotation-based** pointcuts; they're explicit
- Always return the result of `proceed()` from `@Around`

TIMING EVERY @TIMED METHOD

```
@Aspect @Component
class TimingAspect {
    private static final Logger log = LoggerFactory.getLogger(TimingAspect.class);

    @Around("@annotation(com.example.Timed)")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try {
            return pjp.proceed();                    // call the real method
        } finally {
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.info("{} took {} ms", pjp.getSignature().toShortString(), ms);
        }
    }

    @Pointcut("within(com.example..service..*)")
    void services() { }

    @AfterThrowing(pointcut = "services()", throwing = "ex")
    void logError(JoinPoint jp, Exception ex) { log.warn("{} failed", jp, ex); }
}
```

CODE EXPLAINED

- → `@annotation(...)` matches methods carrying `@Timed`, a plain marker annotation
- → `finally` logs the time even when the method throws
- → `@Pointcut` names a reusable expression: `services()`
- → `throwing = "ex"` binds the exception to the parameter

ADVICE TYPES

ANNOTATION	RUNS
`@Before`	Before the call
`@AfterReturning`	After success
`@AfterThrowing`	After an exception
`@After`	Always (finally)
`@Around`	Wraps the call

POINTCUT DESIGNATORS

EXPRESSION	MATCHES
`execution(* *..Svc.*(..))`	By signature
`within(com.x..*)`	Types in package
`@annotation(Ann)`	Annotated method
`@within(Ann)`	Annotated class
`bean(*Service)`	Bean name

JOINPOINT API

METHOD	RETURNS
`getSignature()`	Method signature
`getArgs()`	Arguments
`getTarget()`	Real bean
`proceed()`	Result (`@Around`)
`proceed(args)`	With new args

⚠ COMMON MISTAKES

- Forgetting `@Component` on the aspect; it's never registered
- Not returning `pjp.proceed()` in `@Around`, so callers get `null`
- Over-broad `execution(* *(..))` pointcuts that advise Spring's own beans

✓ TIPS & BEST PRACTICES

- Use a marker annotation (`@Timed`, `@Audited`) as the pointcut
- Use Micrometer's `@Observed` for timing instead of hand-made aspects
- Keep advice short; it runs on every matching call

QUICK SUMMARY

- `@Aspect @Component`: the aspect
- `@Around` + `proceed()`: wrap a call
- `@annotation(X)`: match by annotation
- `@Pointcut`: reusable expression

Controllers & Request Mappings

MAPPING REQUESTS TO METHODS

A **`@RestController`** handles HTTP requests; each method's return value is written to the response body as JSON by default. **`@RequestMapping`** on the class sets a common path prefix, and **`@GetMapping`**, **`@PostMapping`**, **`@PutMapping`**, **`@PatchMapping`** and **`@DeleteMapping`** map individual methods by HTTP method and path.

NOTES

- Return an object → Jackson writes JSON with status 200
- Paths support `{vars}`, `*`, `**` and regex `{id:\d+}`
- Trailing-slash matching is **off**: `/orders/` ≠ `/orders`
- `produces` / `consumes` restrict media types

A CRUD CONTROLLER

```
@RestController
@RequestMapping("/api/orders")
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) { this.service = service; }

    @GetMapping                                  // GET /api/orders
    List<Order> list() { return service.findAll(); }

    @GetMapping("/{id}")                         // GET /api/orders/42
    Order get(@PathVariable long id) { return service.find(id); }

    @PostMapping                                 // POST /api/orders
    @ResponseStatus(HttpStatus.CREATED)          // 201
    Order create(@RequestBody NewOrder body) { return service.create(body); }

    @PutMapping("/{id}")
    Order replace(@PathVariable long id, @RequestBody NewOrder body) { … }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)       // 204
    void delete(@PathVariable long id) { service.delete(id); }
}
```

CODE EXPLAINED

→ Class-level `@RequestMapping` prefixes every method path

→ `@RequestBody` reads JSON into `NewOrder` (Sheet 21)

→ `@ResponseStatus` changes the success status code

→ `void` + 204 sends an empty body

MAPPING ANNOTATIONS

ANNOTATION	HTTP METHOD
`@GetMapping`	GET: read
`@PostMapping`	POST: create
`@PutMapping`	PUT: replace
`@PatchMapping`	PATCH: partial update
`@DeleteMapping`	DELETE: remove

MAPPING ATTRIBUTES

ATTRIBUTE	EXAMPLE
`path / value`	`"/{id}"`
`params`	`"type=pdf"`
`headers`	`"X-Api=1"`
`consumes`	`"application/json"`
`produces`	`"text/csv"`

PATH PATTERNS

PATTERN	MATCHES
`/files/*`	One segment
`/files/**`	Any depth
`/{id}`	Captures `id`
`/{id:\d+}`	Digits only
`/{*path}`	Rest of path

⚠ COMMON MISTAKES

- Using `@Controller` for a JSON API; Spring looks for a view template
- Two methods with the same path + method → ambiguous mapping at startup
- Returning JPA entities directly; lazy fields and cycles break JSON

✓ TIPS & BEST PRACTICES

- Return **records** as DTOs, not entities
- List all mappings at `/actuator/mappings`
- Use `ResponseEntity` when the status depends on the outcome (Sheet 22)

QUICK SUMMARY

- `@RestController`: JSON responses
- `@RequestMapping`: class prefix
- `@GetMapping` …: per method
- `@ResponseStatus`: success code

Reading Request Data

GETTING DATA OUT OF A REQUEST

Handler method **parameters** declare what you need from the request: **`@PathVariable`** for URI segments, **`@RequestParam`** for query strings and form fields, **`@RequestBody`** for a JSON body, plus headers and cookies. Spring converts each value to the parameter's type and returns **400 Bad Request** when a required value is missing or invalid.

NOTES

- `@RequestParam` is required by default; set `required = false` or a default
- Use `Optional<T>` or `@Nullable` for optional values
- A plain object parameter (no annotation) binds query params by name
- `@RequestBody` uses Jackson; only **one** body per request

EVERY KIND OF INPUT

```
@GetMapping("/products/{category}/{id}")
Product show(@PathVariable String category,                  // /products/books/7
             @PathVariable("id") long productId) { … }

@GetMapping("/products")                                     // ?q=lamp&page=2
Page<Product> search(@RequestParam String q,
                     @RequestParam(defaultValue = "0") int page,
                     @RequestParam Optional<String> sort,
                     @RequestParam List<String> tag) { … }   // ?tag=a&tag=b

@PostMapping("/products")
Product create(@RequestBody @Valid NewProduct body,          // JSON body
               @RequestHeader("X-Request-Id") String reqId,
               @CookieValue(name = "cart", required = false) String cart) { … }

record ProductFilter(String q, BigDecimal maxPrice, int page) { }

@GetMapping("/filter")                                       // ?q=x&maxPrice=20
List<Product> filter(ProductFilter f) { … }                  // binds by name
```

CODE EXPLAINED

- → `@PathVariable("id")` maps a differently named segment
- → `defaultValue` implies `required = false`
- → Repeated params bind to `List<String>`
- → `@Valid` runs Bean Validation on the body (Sheet 23)
- → A record parameter is filled from matching query params

PARAMETER ANNOTATIONS

ANNOTATION	READS
`@PathVariable`	URI template var
`@RequestParam`	Query / form field
`@RequestBody`	Body (JSON)
`@RequestHeader`	Header value
`@CookieValue`	Cookie value

INJECTABLE WITHOUT ANNOTATION

TYPE	GIVES
`HttpServletRequest`	Raw request
`Principal`	Current user
`Locale`	Request locale
`MultipartFile`	Uploaded file
`UriComponentsBuilder`	Build URLs

WHAT FAILS

PROBLEM	STATUS
`Missing param`	400
`` `abc` for `int` ``	400
`Malformed JSON`	400
`Wrong Content-Type`	415
`Wrong Accept`	406

⚠ COMMON MISTAKES

- Primitive `int` for an optional param; missing → 400, use `Integer` or a default
- Sending form data to a `@RequestBody` method; it expects JSON
- Expecting `@RequestParam Map` to hold repeated values; use `MultiValueMap`

✓ TIPS & BEST PRACTICES

- Group many query params into a **record** parameter
- Accept `MultipartFile` for uploads; set `spring.servlet.multipart.max-file-size`
- Use `@RequestHeader HttpHeaders headers` to read them all

QUICK SUMMARY

- `@PathVariable`: `/items/{id}`
- `@RequestParam`: `?q=…`
- `@RequestBody`: JSON body
- Record param: bind many query params

Building Responses

CONTROLLING THE RESPONSE

Returning an object gives **200 OK** with a JSON body. When you need a different status, headers or an empty body, return **`ResponseEntity<T>`**, built with static factories such as `ok()`, `created(uri)`, `noContent()` and `notFound()`. For a fixed status on every success, `@ResponseStatus` on the method is shorter.

NOTES

- `ResponseEntity<Void>` for responses with no body
- `created(uri)` sets status 201 and the `Location` header
- `ResponseEntity.of(optional)` → 200 with body or 404
- Errors are better handled centrally (Sheet 24)

RESPONSEENTITY BUILDERS

```
@GetMapping("/{id}")
ResponseEntity<Order> get(@PathVariable long id) {
    return ResponseEntity.of(repo.findById(id));            // 200 or 404
}

@PostMapping
ResponseEntity<Order> create(@RequestBody NewOrder body, UriComponentsBuilder ucb) {
    Order saved = service.create(body);
    URI location = ucb.path("/api/orders/{id}").build(saved.id());
    return ResponseEntity.created(location).body(saved);    // 201 + Location
}

@DeleteMapping("/{id}")
ResponseEntity<Void> delete(@PathVariable long id) {
    return service.delete(id) ? ResponseEntity.noContent().build()   // 204
                              : ResponseEntity.notFound().build();   // 404
}

@GetMapping(value = "/{id}/invoice", produces = MediaType.APPLICATION_PDF_VALUE)
ResponseEntity<byte[]> invoice(@PathVariable long id) {
    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=inv.pdf")
        .cacheControl(CacheControl.noStore())
        .body(pdf.render(id));
}
```

CODE EXPLAINED

- → `UriComponentsBuilder` is pre-filled with the request's host and port
- → `.build()` ends a builder with **no** body
- → `header(...)` / `cacheControl(...)` set response headers
- → `byte[]` with `produces` streams a file download

STATIC FACTORIES

METHOD	STATUS
`ok(body)`	200
`created(uri)`	201 + Location
`accepted()`	202
`noContent()`	204
`notFound()`	404

BUILDER METHODS

METHOD	SETS
`status(code)`	Any status
`header(k, v)`	One header
`contentType(mt)`	Content-Type
`eTag(tag)`	ETag
`body(obj)`	Body, ends chain

OTHER RETURN TYPES

TYPE	USE
`record / POJO`	200 JSON
`void`	200 / `@ResponseStatus`
`Resource`	File download
`byte[]`	Raw bytes
`SseEmitter`	Server-sent events

⚠ COMMON MISTAKES

- Returning `null` for "not found"; the client gets 200 with an empty body
- Building `Location` from a hard-coded host; use `UriComponentsBuilder`
- Wrapping every method in `ResponseEntity` even when status is always 200

✓ TIPS & BEST PRACTICES

- Use `ResponseEntity.of(Optional)` for simple lookups
- Throw a custom exception for errors and map it once (Sheet 24)
- Use `HttpStatus` constants, not raw numbers

QUICK SUMMARY

- `ResponseEntity<T>` : status + headers + body
- `created(uri)` : 201 with Location
- `noContent()` / `notFound()` : no body
- `@ResponseStatus` : fixed status

Bean Validation

VALIDATING INPUT

Add **`spring-boot-starter-validation`** and annotate fields with **Jakarta Bean Validation** constraints like `@NotBlank`, `@Size` and `@Email`. Put **`@Valid`** on a `@RequestBody` (or record parameter) and Spring validates it before your method runs; failures return **400** with field errors. Since Framework 6.1, constraints on `@RequestParam` and `@PathVariable` are checked too, with no extra setup.

NOTES

- Implementation is **Hibernate Validator**
- `@Valid` also cascades into nested objects and list elements
- `null` passes most constraints; add `@NotNull` when required
- Messages can come from `ValidationMessages.properties`

CONSTRAINTS ON A REQUEST DTO

```
record NewCustomer(
    @NotBlank @Size(max = 80)          String name,
    @NotBlank @Email                   String email,
    @Min(18) @Max(130)                 int age,
    @Pattern(regexp = "\\+?[0-9 ]{7,15}") String phone,
    @NotEmpty List<@Valid Address> addresses) { }

record Address(@NotBlank String street, @NotBlank String city) { }

@RestController                                  // no @Validated needed (6.1+)
class CustomerController {

    @PostMapping("/customers")
    Customer create(@RequestBody @Valid NewCustomer body) { … }

    @GetMapping("/customers")
    List<Customer> page(@RequestParam @Min(1) @Max(100) int size) { … }
}
```

CODE EXPLAINED

- → Body failures throw `MethodArgumentNotValidException` → 400
- → Param failures throw `HandlerMethodValidationException` → 400
- → `List<@Valid Address>` validates every element
- → `SkuValidator` implements `ConstraintValidator<Sku, String>`

CUSTOM CONSTRAINT

```
@Constraint(validatedBy = SkuValidator.class) @Target(FIELD) @Retention(RUNTIME)
@interface Sku { String message() default "bad SKU"; Class<?>[] groups() default {};
                 Class<? extends Payload>[] payload() default {}; }
```

COMMON CONSTRAINTS

ANNOTATION	CHECKS
`@NotNull`	Not null
`@NotBlank`	Text has non-space
`@NotEmpty`	Not null or empty
`@Size(min,max)`	Length / count
`@Email`	Email format

NUMBERS & DATES

ANNOTATION	CHECKS
`@Min / @Max`	Integer bounds
`@Positive`	> 0
`@DecimalMax("9.99")`	Decimal bound
`@Past / @Future`	Dates
`@Pattern(regexp)`	Regex match

@VALID VS @VALIDATED

ANNOTATION	USE ON
`@Valid`	Body, nested objects
`@Validated`	Service classes
`@Validated(G.class)`	Validation groups

⚠ COMMON MISTAKES

- Annotating fields but forgetting `@Valid` on the parameter; nothing is checked
- `@Validated` on a controller: errors become 500 `ConstraintViolationException`
- Missing the validation starter; annotations compile but never run

✓ TIPS & BEST PRACTICES

- Validate DTOs at the edge, not JPA entities
- Return field errors in a `ProblemDetail` (Sheet 24)
- Validate config classes too: `@Validated` on `@ConfigurationProperties`

QUICK SUMMARY

- `starter-validation` : enables it
- `@Valid @RequestBody` : check body
- Param constraints: checked (6.1+)
- Fail → 400 with field errors

Exception Handling

TURNING EXCEPTIONS INTO RESPONSES

Handle errors in one place with a **`@RestControllerAdvice`** class whose **`@ExceptionHandler`** methods map exception types to responses. Spring supports **RFC 9457 Problem Details**: return a **`ProblemDetail`** and it's sent as `application/problem+json`. Set **`spring.mvc.problemdetails.enabled=true`** so Spring's own errors use the same format.

NOTES

- `@ExceptionHandler` in a controller applies to that controller only
- Advice handlers apply to **all** controllers (narrow with `basePackages`)
- Throw `ResponseStatusException` for quick one-off errors
- Extend `ResponseEntityExceptionHandler` to customize Spring's built-in errors

GLOBAL ERROR HANDLING

```
@RestControllerAdvice
class ApiErrors {

    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Order not found");
        pd.setType(URI.create("https://shop.example.com/errors/not-found"));
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail invalid(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setProperty("errors", ex.getFieldErrors().stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage()).toList());
        return pd;
    }
}
```

CODE EXPLAINED

→ The status inside `ProblemDetail` becomes the response status

→ Body: `type`, `title`, `status`, `detail`, `instance` (request path)

→ `setProperty()` adds custom fields to the JSON

→ Validation handler lists every failing field

ERROR TOOLS

TOOL	USE
`@RestControllerAdvice`	Global handlers
`@ExceptionHandler`	Map exception type
`ProblemDetail`	RFC 9457 body
`ResponseStatusException`	Quick status error

PROBLEMDETAIL FIELDS

FIELD	MEANING
`type`	URI for error kind
`title`	Short summary
`status`	HTTP status
`detail`	This occurrence
`instance`	Request path

SPRING EXCEPTIONS

EXCEPTION	STATUS
`…ArgumentNotValid`	400
`NoResourceFound`	404
`…MethodNotSupported`	405
`…MediaTypeNotSupported`	415

⚠ COMMON MISTAKES

- Catching exceptions in every controller method instead of one advice class
- Leaking stack traces or SQL in `detail`; log them, return a safe message
- A catch-all `Exception` handler that also swallows 404/405 from Spring

✓ TIPS & BEST PRACTICES

- Use stable `type` URIs so clients can switch on error kind
- Extend `ResponseEntityExceptionHandler` to reformat Spring's errors
- Log 5xx at ERROR, 4xx at DEBUG or WARN

QUICK SUMMARY

- `@RestControllerAdvice`: global
- `@ExceptionHandler(X.class)`: map one
- `ProblemDetail`: standard error JSON
- `problemdetails.enabled`: built-ins too

JSON & Jackson

JSON WITH JACKSON 3

Spring MVC turns objects into JSON (and back) with **Jackson**. **Boot 4 uses Jackson 3**: the core classes moved to the **`tools.jackson`** package and the main type is **`JsonMapper`**, while the annotations kept their `com.fasterxml.jackson.annotation` package. Shape output with annotations on your DTOs and tune defaults with `spring.jackson.*` properties.

NOTES

- Records serialize out of the box: components become fields
- `java.time` types are written as ISO-8601 strings
- Customize the mapper with a `JsonMapperBuilderCustomizer` bean
- `@JsonComponent` became `@JacksonComponent` in Boot 4

SHAPING A DTO

```java
import com.fasterxml.jackson.annotation.*;   // annotations: unchanged package

@JsonInclude(JsonInclude.Include.NON_NULL)   // drop null fields
record OrderView(
    long id,
    @JsonProperty("customer_name") String customerName,
    @JsonFormat(pattern = "yyyy-MM-dd") LocalDate placedOn,
    @JsonIgnore String internalNote,
    List<LineView> lines) { }
```

USING THE MAPPER DIRECTLY

```java
import tools.jackson.databind.json.JsonMapper;   // Jackson 3 package

@Service
class Exporter {
    private final JsonMapper mapper;                  // Boot's configured mapper
    Exporter(JsonMapper mapper) { this.mapper = mapper; }

    String toJson(OrderView v) { return mapper.writeValueAsString(v); }
    OrderView read(String json) { return mapper.readValue(json, OrderView.class); }
}
```

CODE EXPLAINED

→ JSON keys: `id`, `customer_name`, `placedOn`, `lines`

→ `internalNote` never appears in the JSON

→ Jackson 3 exceptions are unchecked: no `throws` needed

→ Tweak Boot's mapper with a `JsonMapperBuilderCustomizer` bean

COMMON ANNOTATIONS

ANNOTATION	EFFECT
`@JsonProperty`	Rename a field
`@JsonIgnore`	Leave it out
`@JsonInclude`	Skip null / empty
`@JsonFormat`	Date / number format
`@JsonAlias`	Extra input names

SPRING.JACKSON.*

PROPERTY	SETS
`…property-inclusion`	`non_null`
`…naming-strategy`	SNAKE_CASE
`json.read.*`	Read features
`json.write.*`	Write features
`time-zone`	Date time zone

JACKSON 2 → 3

JACKSON 2	JACKSON 3
`com.fasterxml…`	`tools.jackson…`
`ObjectMapper`	`JsonMapper`
`JsonProcessing…`	`JacksonException`
`@JsonComponent`	`@JacksonComponent`

⚠ COMMON MISTAKES

- Importing `com.fasterxml.jackson.databind` in Boot 4 code; it's Jackson 2
- Defining your own `ObjectMapper` bean; Boot 4 configures a `JsonMapper`
- Serializing entities with bidirectional relations → infinite recursion

✓ TIPS & BEST PRACTICES

- Use dedicated response records instead of annotating entities
- Set `spring.jackson.default-property-inclusion=non_null` app-wide
- Mid-migration? `spring.jackson.use-jackson2-defaults=true` keeps old output

QUICK SUMMARY

- Boot 4: Jackson 3 (`tools.jackson`)
- `JsonMapper`: inject to (de)serialize
- Annotations: `@JsonProperty`, `@JsonIgnore`
- `*Customizer` bean: tweak defaults

API Versioning

BUILT-IN API VERSIONING

Spring Framework 7 adds first-class **API versioning**. Choose where clients send the version (a **header**, **query parameter**, **path segment** or **media-type parameter**), then add a **`version`** attribute to mappings. A baseline like **`"1.1+"`** matches that version and every later one, so a method keeps serving new versions until you replace it.

NOTES

- Header-only? Set `…apiversion.use.header` in properties
- Other strategies: override `configureApiVersioning()`
- Unsupported version → **400**
- Clients send it with `.apiVersion()` (Sheet 30)

VERSIONED MAPPINGS

```
@RestController
@RequestMapping("/api/accounts")
class AccountController {

    @GetMapping(path = "/{id}", version = "1.0")     // exactly 1.0
    AccountV1 getV1(@PathVariable long id) { … }

    @GetMapping(path = "/{id}", version = "1.1+")    // 1.1, 1.2, 2.0 …
    AccountV2 get(@PathVariable long id) { … }
}

@Configuration
class ApiVersionConfig implements WebMvcConfigurer {
    @Override
    public void configureApiVersioning(ApiVersionConfigurer c) {
        c.useRequestHeader("API-Version")         // API-Version: 1.1
         .useQueryParam("v")                      // or ?v=1.1
         .addSupportedVersions("1.0", "1.1", "2.0")
         .setDefaultVersion("1.0");
    }
}
```

CODE EXPLAINED

→ Both methods share a path; the version picks one

→ `1.1+` is a **baseline**: it keeps matching newer versions

→ Several strategies can be combined; the first one found wins

→ `usePathSegment(1)` reads `/api/{version}/…` instead

OR WITH PROPERTIES

```
spring.mvc.apiversion.use.header=API-Version
spring.mvc.apiversion.default=1.0
```

STRATEGIES

METHOD	CLIENT SENDS
`useRequestHeader`	`API-Version: 1.1`
`useQueryParam`	`?v=1.1`
`usePathSegment(i)`	`/api/v1.1/…`
`useMediaTypeParameter`	`;v=1.1` in Accept

CONFIGURER OPTIONS

METHOD	EFFECT
`setDefaultVersion`	Used when absent
`setVersionRequired`	400 if absent
`addSupportedVersions`	Allow-list

CLIENT SIDE

API	PURPOSE
`ApiVersionInserter`	Where to put it
`.apiVersion("1.1")`	Per request
`@GetExchange(version)`	HTTP interfaces

⚠ COMMON MISTAKES

- Adding `version` to mappings without configuring a strategy; startup fails
- Copying a whole controller per version instead of using baselines
- Path versioning without a `{version}` URI variable in the path

✓ TIPS & BEST PRACTICES

- Prefer header or media-type versioning; URLs stay stable
- `setDeprecationHandler()` sends `Deprecation` / `Sunset` headers
- Keep version DTOs (`AccountV1`) next to the controller

QUICK SUMMARY

- `version = "1.1"` : exact match
- `"1.1+"` : this and later
- `configureApiVersioning()` : strategy
- `apiVersion()` : client side

Filters, Interceptors & CORS

CODE AROUND EVERY REQUEST

A servlet **`Filter`** wraps the whole request, before Spring MVC sees it; extend **`OncePerRequestFilter`** and make it a bean. A **`HandlerInterceptor`** runs inside MVC, around the chosen controller method. **CORS** lets browsers on other origins call your API; configure it per controller with `@CrossOrigin` or globally in a `WebMvcConfigurer`.

NOTES

- Order: filters → `DispatcherServlet` → interceptors → controller
- Spring Security is itself a filter chain (Sheet 45)
- Filter beans apply to **all** URLs unless registered with patterns

FILTER, INTERCEPTOR AND CORS

```
@Component
class RequestIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        res.setHeader("X-Request-Id", UUID.randomUUID().toString());
        chain.doFilter(req, res);                // continue the chain
    }
}

class AuthInterceptor implements HandlerInterceptor {
    public boolean preHandle(HttpServletRequest q, HttpServletResponse s,
                             Object handler) {
        return q.getHeader("X-Api-Key") != null;   // false = stop here
    }
}

@Configuration
class WebConfig implements WebMvcConfigurer {
    public void addInterceptors(InterceptorRegistry r) {
        r.addInterceptor(new AuthInterceptor()).addPathPatterns("/api/**");
    }
    public void addCorsMappings(CorsRegistry r) {
        r.addMapping("/api/**").allowedOrigins("https://app.example.com")
         .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

CODE EXPLAINED

- → `OncePerRequestFilter` runs once even on forwards and error dispatches
- → Forgetting `chain.doFilter()` ends the request silently
- → `preHandle` returning `false` skips the controller
- → CORS answers browser **preflight** `OPTIONS` requests

FILTER VS INTERCEPTOR

ASPECT	FILTER / INTERCEPTOR
Layer	Servlet / Spring MVC
Sees handler	No / Yes
Static files	Yes / Yes
Register	Bean / `addInterceptors`

INTERCEPTOR HOOKS

METHOD	RUNS
`preHandle`	Before controller
`postHandle`	After, before view
`afterCompletion`	Always, at the end

CORS OPTIONS

METHOD	SETS
`allowedOrigins`	Exact origins
`allowedMethods`	HTTP methods
`allowCredentials`	Allow cookies
`maxAge`	Cache preflight

⚠ COMMON MISTAKES

- `allowedOrigins("*")` with `allowCredentials(true)`; browsers reject it
- CORS set in MVC only, while Spring Security blocks the preflight; add `http.cors()`
- Reading the request body in a filter; the controller then gets nothing

✓ TIPS & BEST PRACTICES

- Use `FilterRegistrationBean` to limit a filter to URL patterns or set its order
- Put logging/tracing in filters; handler-aware logic in interceptors
- Use `@CrossOrigin` only for one-off controllers; prefer global config

QUICK SUMMARY

- `OncePerRequestFilter`: servlet level
- `HandlerInterceptor`: around handlers
- `addCorsMappings()`: global CORS
- `@CrossOrigin`: per controller

Thymeleaf Basics

SERVER-RENDERED HTML

For server-side HTML, use a plain **`@Controller`** (not `@RestController`): the method adds data to the **`Model`** and returns a **view name**, which Boot resolves to `templates/{name}.html` and renders with **Thymeleaf**. Thymeleaf templates are valid HTML; dynamic parts live in **`th:*`** attributes, so files open fine in a browser.

NOTES

- Add `spring-boot-starter-thymeleaf`
- `${...}` reads model attributes; `@{...}` builds URLs
- `th:text` escapes HTML; `th:utext` doesn't
- DevTools disables the template cache for live reload

CONTROLLER

```
@Controller
@RequestMapping("/products")
class ProductPageController {
    private final ProductService service;
    ProductPageController(ProductService s) { this.service = s; }

    @GetMapping
    String list(Model model) {
        model.addAttribute("products", service.findAll());
        return "products/list";              // templates/products/list.html
    }
}
```

CODE EXPLAINED

- → The return value is a **view name**, not a body
- → Placeholder text (`Name`) is replaced at render time
- → `stat` gives `index` , `count` , `odd` , `first` , `last`
- → `@{/products/{id}(id=…)}` fills the path variable

TEMPLATES/PRODUCTS/LIST.HTML

```
<html xmlns:th="http://www.thymeleaf.org">
<body>
  <h1 th:text="${#lists.size(products)} + ' products'">0 products</h1>
  <p th:if="${products.isEmpty()}">Nothing here yet.</p>
  <ul>
    <li th:each="p, stat : ${products}" th:classappend="${stat.odd} ? 'odd'">
      <a th:href="@{/products/{id}(id=${p.id})}" th:text="${p.name}">Name</a>
      <span th:text="${#numbers.formatCurrency(p.price)}">$0.00</span>
    </li>
  </ul>
</body>
</html>
```

CORE TH: ATTRIBUTES

ATTRIBUTE	DOES
th:text	Set escaped text
th:each	Loop
th:if / th:unless	Conditional
th:href / th:src	Build URL
th:replace	Insert fragment

EXPRESSIONS

SYNTAX	MEANING
${x}	Model variable
*{field}	On th:object
@{/path}	Context-aware URL
#{key}	i18n message
~{frag}	Fragment ref

UTILITY OBJECTS

OBJECT	EXAMPLE
#strings	#strings.toUpperCase(s)
#lists	#lists.size(l)
#temporals	#temporals.format(d)
#numbers	#numbers.formatDecimal

⚠ COMMON MISTAKES

- Using `@RestController` , which returns the string `"products/list"` as the body
- A leading slash in the view name (`"/products/list"`) breaks some setups
- Using `th:utext` with user input; that's an XSS hole

✓ TIPS & BEST PRACTICES

- Share headers and footers with `th:fragment` + `th:replace`
- Use `#temporals` for `java.time` values
- Add `htmx` for interactivity without a SPA

QUICK SUMMARY

- `@Controller` + `Model` : view data
- Return view name → `templates/`
- `th:text` , `th:each` , `th:if`
- `@{...}` : URLs · `${...}` : data

Thymeleaf Forms & Static Resources

HTML FORMS THE SPRING WAY

A form binds to a **form-backing object**: `th:object` names it and `th:field` connects each input to a property. On submit, Spring binds the fields into a **`@ModelAttribute`** parameter, `@Valid` checks it, and **`BindingResult`** holds the errors. On success, **redirect** (Post/Redirect/Get) and pass a flash message with **`RedirectAttributes`**.

NOTES

- `BindingResult` must come **right after** the validated parameter
- `th:field` sets `id`, `name` and `value` for you
- Thymeleaf adds the CSRF token to `th:action` forms automatically
- Static files in `static/` are served from `/`

GET FORM, POST HANDLER

```
@GetMapping("/new")
String form(Model model) {
    model.addAttribute("product", new ProductForm());
    return "products/form";
}

@PostMapping
String save(@Valid @ModelAttribute("product") ProductForm form,
            BindingResult result, RedirectAttributes flash) {
    if (result.hasErrors()) return "products/form";   // re-show with errors
    service.create(form);
    flash.addFlashAttribute("msg", "Product saved");
    return "redirect:/products";                      // PRG
}
```

CODE EXPLAINED

- → The GET puts an empty object in the model for `th:object`
- → `*{name}` is shorthand for `${product.name}`
- → `th:errors` prints that field's validation messages
- → Flash attributes survive exactly one redirect

TEMPLATES/PRODUCTS/FORM.HTML

```
<form th:action="@{/products}" th:object="${product}" method="post">
  <input th:field="*{name}" />
  <p th:if="${#fields.hasErrors('name')}" th:errors="*{name}">err</p>
  <input th:field="*{price}" type="number" step="0.01" />
  <button>Save</button>
</form>
<p th:text="${msg}" th:if="${msg}"></p>          <!-- on list page -->
<link th:href="@{/css/app.css}" rel="stylesheet"> <!-- static/css/app.css -->
```

HANDLER RETURN VALUES

RETURN	RESULT
`"view"`	Render template
`"redirect:/x"`	302 to /x
`"forward:/x"`	Internal forward
`ModelAndView`	View + model

#FIELDS HELPERS

EXPRESSION	GIVES
`hasErrors('f')`	Field has errors
`hasErrors('*')`	Any errors
`errors('f')`	List of messages
`hasGlobalErrors()`	Object-level errors

STATIC RESOURCES

LOCATION	URL
`static/app.css`	`/app.css`
`public/x.png`	`/x.png`
`…static-path-pattern`	Change prefix
`…cache.period`	Cache header

⚠ COMMON MISTAKES

- Returning the view after a successful POST; refresh resubmits the form
- Putting `BindingResult` after another parameter; Spring throws instead
- Forgetting to add the form object on GET; `th:object` fails

✓ TIPS & BEST PRACTICES

- Use a separate form class, not the entity, for user input
- Use `@ModelAttribute` methods for dropdown data every view needs
- Turn on the resource chain's content strategy for cache-busting asset URLs

QUICK SUMMARY

- `th:object` + `th:field`: bind form
- `@Valid` + `BindingResult`: errors
- `redirect:` + flash: PRG
- `static/`: served at `/`

RestClient

CALLING HTTP APIS

`RestClient` is Spring's synchronous HTTP client with a fluent API. Inject Boot's pre-configured **`RestClient.Builder`**, set a base URL and default headers, and build one client per remote service. Each call chains **method → URI → headers/body → `retrieve()` → `body()`**; 4xx/5xx responses throw unless handled by **`onStatus()`**.

NOTES

- The builder is a **prototype** bean: each injection is a fresh copy
- `RestTemplate` still works but is in maintenance mode
- Global timeouts: `spring.http.clients.connect-timeout` / `read-timeout`

A CLIENT FOR ONE SERVICE

```
@Component
class GitHubClient {
    private final RestClient http;

    GitHubClient(RestClient.Builder builder) {
        this.http = builder.baseUrl("https://api.github.com")
            .defaultHeader(HttpHeaders.ACCEPT, "application/vnd.github+json")
            .build();
    }

    Repo repo(String owner, String name) {
        return http.get()
            .uri("/repos/{owner}/{name}", owner, name)       // URI vars
            .retrieve()
            .onStatus(s -> s.value() == 404,
                      (req, res) -> { throw new RepoNotFound(name); })
            .body(Repo.class);                               // JSON → record
    }

    ResponseEntity<Void> star(String owner, String name, String token) {
        return http.put().uri("/user/starred/{o}/{n}", owner, name)
            .headers(h -> h.setBearerAuth(token))
            .retrieve().toBodilessEntity();                 // status only
    }
}
```

CODE EXPLAINED

→ `{owner}` and `{name}` are filled and URL-encoded in order

→ `onStatus` maps a status to your own exception

→ `body(Repo.class)` reads JSON with Boot's `JsonMapper`

REQUEST CHAIN

STEP	METHODS
Method	`get()` `post()` `put()` …
URI	`uri(t, vars…)`
Body	`body(obj)`, `header()`
Send	`retrieve()` / `exchange()`

AFTER RETRIEVE()

METHOD	RETURNS
`body(T.class)`	Body only
`toEntity(T.class)`	ResponseEntity<T>
`toBodilessEntity`	Status + headers
`onStatus(…)`	Custom errors

BUILDER OPTIONS

METHOD	SETS
`baseUrl`	Common prefix
`defaultHeader`	Every request
`requestInterceptor`	Wrap each call
`apiVersionInserter`	Version header

⚠ COMMON MISTAKES

- Building a new `RestClient` per call; build once and reuse (it's thread-safe)
- Concatenating query strings by hand; use URI variables or `uriBuilder`
- No timeouts set; a slow server can block request threads

✓ TIPS & BEST PRACTICES

- Use HTTP interface clients for many endpoints (Sheet 31)
- Use `builder.clone()` to derive a second client from one builder
- Log calls with a `requestInterceptor`

QUICK SUMMARY

- `RestClient.Builder` : inject, then build
- `get().uri().retrieve().body()`
- `onStatus()` : map errors
- `toEntity()` : status + headers too

HTTP Interface Clients

DECLARATIVE HTTP CLIENTS

An **HTTP interface** is a Java interface whose methods carry **`@GetExchange`**, **`@PostExchange`** and friends; Spring generates the implementation backed by `RestClient`. In **Boot 4**, **`@ImportHttpServices`** registers interfaces as beans in named **groups**, and each group is configured with `spring.http.serviceclient.<group>.*` properties.

NOTES

- Method parameters use the same annotations as controllers
- Return types: the body, `ResponseEntity<T>`, or `void`
- Errors throw `RestClientResponseException` subclasses
- Works with `WebClient` too, returning `Mono` / `Flux`

INTERFACE + REGISTRATION

```java
@HttpExchange("/repos")
public interface RepoApi {

    @GetExchange("/{owner}/{repo}")
    Repo get(@PathVariable String owner, @PathVariable String repo);

    @GetExchange("/{owner}/{repo}/issues")
    List<Issue> issues(@PathVariable String owner, @PathVariable String repo,
                       @RequestParam(defaultValue = "open") String state);

    @PostExchange("/{owner}/{repo}/issues")
    Issue create(@PathVariable String owner, @PathVariable String repo,
                 @RequestBody NewIssue issue);
}

@SpringBootApplication
@ImportHttpServices(group = "github", types = RepoApi.class)
class App { }
```

CODE EXPLAINED

- → `@HttpExchange` on the type sets a shared path prefix
- → `group = "github"` links the interface to its config block
- → Inject `RepoApi` like any bean; there's no implementation class
- → Before Boot 4: build it with `HttpServiceProxyFactory` yourself

GROUP CONFIG

```yaml
spring.http.serviceclient.github:
  base-url: https://api.github.com
  read-timeout: 5s
```

EXCHANGE ANNOTATIONS

ANNOTATION	HTTP
`@GetExchange`	GET
`@PostExchange`	POST
`@PutExchange`	PUT
`@DeleteExchange`	DELETE

PARAMETER ANNOTATIONS

ANNOTATION	SENDS
`@PathVariable`	URI variable
`@RequestParam`	Query param
`@RequestHeader`	Header
`@RequestBody`	JSON body

@IMPORTHTTPSERVICES

ATTRIBUTE	PURPOSE
`group`	Config group name
`types`	Interfaces to import
`basePackages`	Scan for interfaces
`clientType`	RestClient / WebClient

⚠ COMMON MISTAKES

- Annotating the interface with `@Component`; it's not a class Spring can build
- Forgetting `base-url`, so relative paths have no host
- Using controller annotations like `@GetMapping` instead of `@GetExchange`

✓ TIPS & BEST PRACTICES

- One group per remote service, with its own timeouts and headers
- Customize a group in code with an `HttpServiceGroupConfigurer` bean
- Test with `MockRestServiceServer` or WireMock

QUICK SUMMARY

- `@HttpExchange` interface = client
- `@GetExchange` …: per method
- `@ImportHttpServices`: register beans
- `spring.http.serviceclient.<group>`

WebClient

THE REACTIVE CLIENT

`WebClient` is the non-blocking HTTP client from Spring WebFlux. Calls return **`Mono<T>`** (zero or one value) or **`Flux<T>`** (a stream) instead of the value itself, so many requests can run concurrently on few threads. In a regular Spring MVC app, **`RestClient` is simpler**; reach for WebClient when you're on WebFlux or need streaming.

NOTES

- Needs `spring-boot-starter-webflux` (MVC apps stay on Tomcat)
- Inject Boot's `WebClient.Builder`, like `RestClient.Builder`
- Nothing happens until something **subscribes** (or you `block()`)
- Never `block()` inside a WebFlux request thread

MONO, FLUX AND TIMEOUTS

```
@Component
class WeatherClient {
    private final WebClient http;
    WeatherClient(WebClient.Builder b) {
        this.http = b.baseUrl("https://api.weather.example").build();
    }

    Mono<Forecast> forecast(String city) {
        return http.get().uri("/forecast?city={c}", city)
            .retrieve()
            .bodyToMono(Forecast.class)
            .timeout(Duration.ofSeconds(3))                    // per call
            .onErrorResume(WebClientResponseException.NotFound.class,
                           e -> Mono.empty());
    }

    Flux<Alert> alerts() {                                     // stream
        return http.get().uri("/alerts").retrieve().bodyToFlux(Alert.class);
    }
}

Mono.zip(client.forecast("Oslo"), client.forecast("Rome"))
    .map(t -> List.of(t.getT1(), t.getT2()))
    .block(Duration.ofSeconds(5));                             // MVC only
```

CODE EXPLAINED

- → `bodyToMono` / `bodyToFlux` decode JSON lazily
- → `timeout()` fails the `Mono` if no result in time
- → `onErrorResume` turns a 404 into an empty result
- → `Mono.zip` runs both calls at once and waits for both

RESTCLIENT VS WEBCLIENT

ASPECT	REST / WEB
`Style`	Blocking / Reactive
`Returns`	T / `Mono<T>`
`Starter`	webmvc / webflux
`Streaming`	Limited / Native

COMMON OPERATORS

OPERATOR	DOES
`map`	Transform value
`flatMap`	Chain async call
`zip`	Combine results
`retryWhen`	Retry with backoff
`block()`	Wait (non-reactive)

ERROR HANDLING

METHOD	USE
`onStatus`	Map status → error
`onErrorResume`	Fallback value
`timeout`	Fail if slow

⚠ COMMON MISTAKES

- Adding WebFlux just for WebClient when `RestClient` would do
- Calling `block()` in a reactive pipeline; it can deadlock the event loop
- Building a `Mono` and never subscribing; the request is never sent

✓ TIPS & BEST PRACTICES

- With virtual threads, blocking `RestClient` code scales well too
- Use `Mono.zip` / `Flux.merge` for fan-out calls
- Set connector timeouts via `spring.http.clients.*` properties

QUICK SUMMARY

- `WebClient`: non-blocking client
- `bodyToMono` / `bodyToFlux`
- Lazy: runs on subscribe / `block()`
- Prefer `RestClient` in MVC apps

DataSource Configuration

CONNECTING TO A DATABASE

Add a JDBC starter (`jdbc` or `data-jpa`) plus a **driver**, set **`spring.datasource.url`**, username and password, and Boot creates a pooled **DataSource** using **HikariCP**. With an **embedded** database such as H2 on the classpath and no URL, Boot starts an in-memory database for you. **Docker Compose** or **Testcontainers** can provide a real database without manual setup.

NOTES

- The driver class is detected from the URL
- Pool settings live under `spring.datasource.hikari.*`
- Keep credentials in env vars: `SPRING_DATASOURCE_PASSWORD`
- `@ServiceConnection` fills the URL for Testcontainers (Sheet 56)

APPLICATION.YML

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/shop
    username: shop
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 10        # default 10
      minimum-idle: 2
      connection-timeout: 5s       # wait for a free connection
      max-lifetime: 30m
```

COMPOSE.YAML (WITH DOCKER-COMPOSE STARTER)

```yaml
services:
  postgres:
    image: postgres:17
    environment:
      POSTGRES_DB: shop
      POSTGRES_PASSWORD: secret
```

DRIVER

```
runtimeOnly("org.postgresql:postgresql")
```

CODE EXPLAINED

- → Boot reads `url`, `username`, `password` to build the pool
- → Pool size: match DB limits ÷ number of app instances
- → With the compose starter, Boot starts the container and sets the URL
- → Drivers are `runtimeOnly`: your code uses only JDBC interfaces

JDBC URLS

DATABASE	URL PREFIX
PostgreSQL	`jdbc:postgresql://h/db`
MySQL	`jdbc:mysql://h/db`
SQL Server	`jdbc:sqlserver://h`
H2 memory	`jdbc:h2:mem:test`
H2 file	`jdbc:h2:file:./data`

HIKARI SETTINGS

PROPERTY	MEANING
`maximum-pool-size`	Max connections
`minimum-idle`	Idle kept open
`connection-timeout`	Wait for one
`idle-timeout`	Close idle after
`max-lifetime`	Recycle after

H2 FOR DEVELOPMENT

PROPERTY	VALUE
`h2.console.enabled`	`true`
`h2.console.path`	`/h2-console`
`datasource.url`	`jdbc:h2:mem:db`
`user / password`	`sa` / empty

⚠ COMMON MISTAKES

- A huge pool (100+); the database becomes the bottleneck
- Testing only on H2 while production runs Postgres; SQL dialects differ
- Committing the production password in `application.yml`

✓ TIPS & BEST PRACTICES

- Use Testcontainers or Docker Compose to develop against the real database
- Watch pool metrics: `hikaricp.connections.pending` in Actuator
- Set `spring.datasource.hikari.pool-name` for clearer logs

QUICK SUMMARY

- `spring.datasource.url`: connect
- HikariCP: default pool
- Driver: `runtimeOnly`
- Compose / Testcontainers: real DB

JdbcClient & JdbcTemplate

PLAIN SQL WITH JDBCCLIENT

`JdbcClient` (Framework 6.1+) is the modern, fluent way to run SQL without JPA: `sql()` → `param()` → `query()` / `update()`. It maps rows to **records** by column name and supports named or positional parameters. Boot auto-configures one when a `DataSource` exists; the older `JdbcTemplate` is still available.

NOTES

- Named params `:id` or positional `?` — don't mix them
- `query(Class)` maps `snake_case` columns to `camelCase` fields
- Every call joins the current `@Transactional` transaction

REPOSITORY WITH JDBCCLIENT

```
record Customer(long id, String name, String email) { }

@Repository
class CustomerDao {
    private final JdbcClient jdbc;
    CustomerDao(JdbcClient jdbc) { this.jdbc = jdbc; }

    Optional<Customer> find(long id) {
        return jdbc.sql("SELECT * FROM customer WHERE id = :id")
            .param("id", id)
            .query(Customer.class)                // row → record
            .optional();                          // 0 or 1 row
    }

    List<Customer> byName(String prefix) {
        return jdbc.sql("SELECT * FROM customer WHERE name LIKE ?")
            .param(prefix + "%").query(Customer.class).list();
    }

    long insert(String name, String email) {
        var keys = new GeneratedKeyHolder();
        jdbc.sql("INSERT INTO customer(name, email) VALUES (:n, :e)")
            .param("n", name).param("e", email)
            .update(keys, "id");                  // capture generated id
        return keys.getKey().longValue();
    }
}
```

CODE EXPLAINED

- → `optional()` returns `Optional.empty()` for no row
- → `single()` throws if the row count isn't exactly 1
- → `update()` returns rows affected; with a `KeyHolder` it captures ids
- → `query(Integer.class).single()` reads a `count(*)`

RESULT METHODS

METHOD	RETURNS
`list()`	`List<T>`
`single()`	Exactly one or error
`optional()`	`Optional<T>`
`stream()`	`Stream<T>` (close it)

PARAMETERS

METHOD	BINDS
`param(v)`	Next ?
`param("n", v)`	`:n`
`params(map)`	All named
`paramSource(o)`	Record fields

JDBCTEMPLATE EQUIVALENT

JDBCTEMPLATE	JDBCCLIENT
`queryForObject`	`query().single()`
`query + RowMapper`	`query(rm).list()`
`update`	`update()`
`batchUpdate`	(use JdbcTemplate)

⚠ COMMON MISTAKES

- String-concatenating values into SQL; always bind parameters
- `single()` on a lookup that may find nothing; use `optional()`
- Leaving a `stream()` open; wrap it in try-with-resources

✓ TIPS & BEST PRACTICES

- Use `paramSource(record)` to bind every field of an object at once
- Keep SQL in text blocks (`"""`) for readability
- Use `JdbcTemplate.batchUpdate()` for large bulk inserts

QUICK SUMMARY

- `jdbc.sql(...).param(...)`
- `.query(T.class).list()` / `optional()`
- `.update()`: insert / update / delete
- `DataAccessException`: unchecked errors

Schema Init & Migrations

CREATING AND EVOLVING THE SCHEMA

For anything beyond a demo, manage the schema with a **migration tool**. **Flyway** runs versioned SQL files from `db/migration` in order and records each one in `flyway_schema_history`, so every environment ends up identical. **Liquibase** does the same with XML/YAML/SQL changelogs. For quick prototypes, `schema.sql` / `data.sql` or Hibernate's **`ddl-auto`** can create tables.

NOTES

- Boot 4: add `spring-boot-starter-flyway` (or `-liquibase`)
- Migrations run at startup, **before** JPA starts
- Never edit an applied migration; add a new one
- `ddl-auto=validate` checks entities against the real schema

FLYWAY MIGRATION FILES

```sql
-- src/main/resources/db/migration/V1__create_customer.sql
CREATE TABLE customer (
    id         BIGSERIAL PRIMARY KEY,
    name       VARCHAR(80)  NOT NULL,
    email      VARCHAR(120) NOT NULL UNIQUE,
    created_at TIMESTAMP    NOT NULL DEFAULT now()
);

-- V2__add_customer_phone.sql
ALTER TABLE customer ADD COLUMN phone VARCHAR(20);

-- R__customer_view.sql   (repeatable: re-runs when it changes)
CREATE OR REPLACE VIEW active_customer AS SELECT * FROM customer;
```

CODE EXPLAINED

- → `V1__` / `V2__`: version, **two** underscores, description
- → Flyway runs pending versions in order, once each
- → `R__` files re-run whenever their checksum changes
- → `validate` fails startup if an entity doesn't match a table

APPLICATION.YML

```yaml
spring:
  flyway:
    locations: classpath:db/migration
    baseline-on-migrate: true      # adopt an existing database
  jpa.hibernate.ddl-auto: validate   # never let Hibernate change prod
```

DDL-AUTO VALUES

VALUE	DOES
none	Nothing (prod default)
validate	Check, don't change
update	Add missing columns
create	Drop + create
create-drop	…and drop on exit

SCRIPT INIT

PROPERTY	EFFECT
sql.init.mode	always / never
schema.sql	DDL at startup
data.sql	Seed rows
…defer-datasource…	Run after JPA

SPRING.FLYWAY.*

PROPERTY	PURPOSE
enabled	Turn on/off
locations	Script folders
baseline-version	Start version
clean-disabled	Block clean
validate-on-migrate	Checksum check

⚠ COMMON MISTAKES

- Editing `V1__…sql` after it ran; checksum mismatch fails every startup
- `ddl-auto=update` in production; it never drops or renames safely
- Mixing Flyway with `schema.sql`; both try to create the same tables

✓ TIPS & BEST PRACTICES

- Test migrations against the real database with Testcontainers
- Name files by intent: `V7__add_order_status_index.sql`
- Keep data fixes in migrations too, so every environment gets them

QUICK SUMMARY

- `V#__desc.sql`: versioned migration
- `starter-flyway`: runs at startup
- `ddl-auto=validate`: check only
- `schema.sql` / `data.sql`: prototypes

JPA Entities

MAPPING CLASSES TO TABLES

A **JPA entity** is a class annotated with **`@Entity`** whose instances map to table rows. It needs an **`@Id`**, a no-arg constructor (can be `protected`) and non-final fields. **Hibernate** (version 7 in Boot 4) implements JPA: it generates SQL, tracks changes to loaded entities and writes them back when the transaction commits.

NOTES

- Packages are `jakarta.persistence.*`
- Entities can't be records; use records for DTOs
- Unannotated fields are mapped by default; `@Transient` skips one
- Table/column names become `snake_case` by default

AN ENTITY

```java
@Entity
@Table(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 80)
    private String name;
    @Column(unique = true)
    private String email;
    @Enumerated(EnumType.STRING)        // store "GOLD", not 2
    private Tier tier = Tier.BASIC;
    @Embedded
    private Address address;            // street, city columns inline
    @Version
    private int version;                // optimistic locking

    protected Customer() { }            // for JPA
    public Customer(String name, String email) { … }
}

@Embeddable
public record Address(String street, String city) { }
```

CODE EXPLAINED

- → `IDENTITY` uses the database's auto-increment column
- → `EnumType.STRING` survives reordering the enum
- → `@Embeddable` (records allowed) adds columns to the same table
- → `@Version` makes concurrent updates fail instead of overwrite

FIELD ANNOTATIONS

ANNOTATION	EFFECT
`@Id`	Primary key
`@GeneratedValue`	DB-generated key
`@Column`	Name, length, nullable
`@Enumerated`	Enum storage
`@Transient`	Not persisted

ID STRATEGIES

STRATEGY	USES
`IDENTITY`	Auto-increment
`SEQUENCE`	DB sequence (batching)
`UUID`	`UUID` keys
`AUTO`	Provider decides

TYPE MAPPING

JAVA	COLUMN
`String`	VARCHAR(255)
`BigDecimal`	NUMERIC
`LocalDateTime`	TIMESTAMP
`boolean`	BOOLEAN
`@Lob String`	TEXT / CLOB

⚠ COMMON MISTAKES

- `equals` / `hashCode` on all fields or on a null `id`; sets break after save
- `EnumType.ORDINAL` (the default); reordering the enum corrupts data
- Exposing entities in REST responses; lazy fields and cycles break JSON

✓ TIPS & BEST PRACTICES

- Base `equals` / `hashCode` on the id with a constant hash, or a natural key
- Use `SEQUENCE` on Postgres so Hibernate can batch inserts
- Keep entities in the domain layer; map to DTO records at the edges

QUICK SUMMARY

- `@Entity` + `@Id`: a mapped table
- `protected` no-arg constructor
- `EnumType.STRING`: safe enums
- `@Version`: optimistic locking

Entity Relationships

ASSOCIATIONS BETWEEN ENTITIES

Relationships map foreign keys to object references. **`@ManyToOne`** is the owning side with the FK column; **`@OneToMany(mappedBy = …)`** is its inverse; **`@ManyToMany`** uses a join table. The side **without** `mappedBy` owns the relationship, and only changes to the owning side are written to the database.

NOTES

- Make `@ManyToOne` **LAZY**; its default is EAGER
- `@OneToMany` and `@ManyToMany` are LAZY by default
- Keep both sides in sync with helper methods
- Use `Set` for `@ManyToMany` to avoid duplicate-delete issues

ORDER ↔ ORDERLINE

```
@Entity
class Order {
    @Id @GeneratedValue private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "customer_id")             // FK column
    private Customer customer;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<OrderLine> lines = new ArrayList<>();

    @ManyToMany
    @JoinTable(name = "order_tag",
        joinColumns = @JoinColumn(name = "order_id"),
        inverseJoinColumns = @JoinColumn(name = "tag_id"))
    private Set<Tag> tags = new HashSet<>();

    void addLine(OrderLine l)    { lines.add(l); l.setOrder(this); }
    void removeLine(OrderLine l) { lines.remove(l); l.setOrder(null); }
}

@Entity
class OrderLine {
    @Id @GeneratedValue private Long id;
    @ManyToOne(fetch = FetchType.LAZY) private Order order;   // owning side
}
```

CODE EXPLAINED

→ `mappedBy = "order"` names the field on `OrderLine` that owns it

→ `CascadeType.ALL`: saving an `Order` saves its lines

→ `orphanRemoval`: removing a line from the list deletes the row

→ `addLine` sets both sides, so memory and DB agree

RELATIONSHIP TYPES

ANNOTATION	DEFAULT FETCH
`@ManyToOne`	EAGER (set LAZY)
`@OneToOne`	EAGER (set LAZY)
`@OneToMany`	LAZY
`@ManyToMany`	LAZY

CASCADETYPE

VALUE	PROPAGATES
`PERSIST`	Save new
`MERGE`	Merge detached
`REMOVE`	Delete
`ALL`	Everything

JOIN ANNOTATIONS

ANNOTATION	SETS
`@JoinColumn`	FK column name
`@JoinTable`	Link table
`@OrderBy("name")`	Collection order
`@MapsId`	Share parent's PK

⚠ COMMON MISTAKES

- Setting only `order.getLines().add(line)`; the FK isn't written
- `CascadeType.REMOVE` on `@ManyToMany`, deleting shared rows
- Leaving `@ManyToOne` EAGER; every load pulls the parent too

✓ TIPS & BEST PRACTICES

- Prefer unidirectional `@ManyToOne` unless you need the collection
- Model a join table with extra columns as its own entity
- Initialize collections at declaration to avoid `null`

QUICK SUMMARY

- `@ManyToOne`: owning side + FK
- `mappedBy`: inverse side
- `cascade` + `orphanRemoval`: children
- `FetchType.LAZY`: default choice

Repositories

REPOSITORIES

Declare an **interface** extending **`JpaRepository<Entity, IdType>`** and Spring Data generates the implementation at startup: CRUD, paging, sorting and flushing with no code. Add your own query methods by name (Sheet 39) or with `@Query` (Sheet 40). Boot finds repositories under the main class's package automatically.

NOTES

- Repository methods are `@Transactional` (reads are read-only)
- `save()` inserts when id is `null`, otherwise merges
- `findById()` returns `Optional<T>`
- `List*` interfaces return `List` instead of `Iterable`

DECLARING AND USING A REPOSITORY

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    Optional<Customer> findByEmail(String email);      // derived query
}

@Service
class CustomerService {
    private final CustomerRepository repo;
    CustomerService(CustomerRepository repo) { this.repo = repo; }

    Customer register(String name, String email) {
        return repo.save(new Customer(name, email));   // INSERT, id set
    }

    Customer get(long id) {
        return repo.findById(id)
            .orElseThrow(() -> new CustomerNotFound(id));
    }

    void cleanup(List<Long> ids) {
        repo.deleteAllByIdInBatch(ids);                // one DELETE
    }
}
```

CODE EXPLAINED

→ No implementation class: Spring Data creates a proxy

→ `save()` returns the managed entity; use the returned value

→ `orElseThrow` turns a missing row into your exception

→ `…InBatch` methods skip loading entities first

INTERFACE HIERARCHY

INTERFACE	ADDS
`Repository`	Marker only
`CrudRepository`	save, find, delete
`ListCrudRepository`	Returns `List`
`PagingAndSorting…`	`Pageable`, `Sort`
`JpaRepository`	flush, batch, getReference

INHERITED METHODS

METHOD	DOES
`save / saveAll`	Insert or update
`findById`	`Optional<T>`
`findAll`	All rows
`deleteById`	Load + delete
`getReferenceById`	Lazy proxy, no SELECT

CUSTOMIZING

OPTION	USE
`@EnableJpaRepositories`	Non-default packages
`Fragment interface`	Custom method impl
`@NoRepositoryBean`	Shared base interface
`@RepositoryDefinition`	Pick methods

⚠ COMMON MISTAKES

- Ignoring the return value of `save()` for detached entities
- `findAll()` on a big table; use paging (Sheet 41)
- Calling `save()` after changing a managed entity; dirty checking already saves it

✓ TIPS & BEST PRACTICES

- Expose only the methods you need by extending `Repository` directly
- Use `getReferenceById` to set a FK without loading the parent
- Put business rules in services, not repository default methods

QUICK SUMMARY

- `extends JpaRepository<T, ID>`
- `save`, `findById`, `deleteById`
- Generated proxy, no impl class
- `ListCrudRepository`: `List` results

Derived Query Methods

QUERIES FROM METHOD NAMES

Spring Data parses repository method names into queries. A name is a **subject** (`find…By`, `count…By`, `exists…By`, `delete…By`) followed by **predicates** on entity properties joined with `And` / `Or`, plus optional `OrderBy` and limits such as `Top10`. Invalid names fail at **startup**, so typos are caught early.

NOTES

- Property paths traverse relations: `findByCustomerEmail`
- Return `List`, `Optional`, `Stream`, `Page` or `Slice`
- `IgnoreCase` applies to one property; `AllIgnoreCase` to all
- Long names are a hint to switch to `@Query`

DERIVED QUERY METHODS

```
interface OrderRepository extends JpaRepository<Order, Long> {

    List<Order> findByStatus(Status status);

    List<Order> findByStatusAndTotalGreaterThan(Status s, BigDecimal min);

    List<Order> findByCustomerEmailIgnoreCase(String email);   // join

    List<Order> findByCreatedAtBetween(Instant from, Instant to);

    List<Order> findByNoteContaining(String text);             // %text%

    List<Order> findByStatusInOrderByCreatedAtDesc(Collection<Status> s);

    Optional<Order> findFirstByCustomerIdOrderByCreatedAtDesc(long id);

    List<Order> findTop5ByOrderByTotalDesc();

    boolean existsByCustomerIdAndStatus(long customerId, Status status);

    long countByStatus(Status status);

    @Transactional
    long deleteByStatusAndCreatedAtBefore(Status s, Instant cutoff);
}
```

CODE EXPLAINED

- → `CustomerEmail` walks `order.customer.email` with a join
- → `Containing` wraps the value in `%…%` for `LIKE`
- → `First` / `Top5` add a `LIMIT`
- → Derived `delete` loads each entity, then deletes (fires callbacks)

PREDICATE KEYWORDS

KEYWORD	SQL
`Is / Equals`	`= ?`
`Not`	`<> ?`
`LessThan / Before`	`< ?`
`Between`	`BETWEEN ? AND ?`
`In / NotIn`	`IN (…)`

STRINGS & NULLS

KEYWORD	SQL
`Like`	`LIKE ?` (you add %)
`StartingWith`	`LIKE ?%`
`Containing`	`LIKE %?%`
`IsNull`	`IS NULL`
`True / False`	`= true`

SUBJECTS

PREFIX	RETURNS
`find…By`	Entities
`count…By`	`long`
`exists…By`	`boolean`
`delete…By`	Count or list
`findDistinct…By`	No duplicates

⚠ COMMON MISTAKES

- Misspelled property in the name; startup fails with `No property … found`
- `findByName(null)` expecting `IS NULL`; use `findByNameIsNull()`
- Derived `deleteBy` on many rows; it loads each one first

✓ TIPS & BEST PRACTICES

- Switch to `@Query` once a name passes ~4 conditions
- Return `Optional<T>` for single results that may be missing
- Use `@Modifying @Query` for bulk deletes

QUICK SUMMARY

- `findBy…And…` : AND conditions
- `OrderBy…Desc` : sorting
- `Top` / `First` : limits
- `countBy` / `existsBy` : aggregates

@Query & Projections

WRITING YOUR OWN QUERIES

`@Query` puts JPQL (or native SQL with `nativeQuery = true`) on a repository method. Bind values with **named parameters** (`:email`). Add **`@Modifying`** for UPDATE / DELETE. Return **projections** to fetch only the columns you need: an interface with getters, or a **record** (DTO) built by the query.

NOTES

- JPQL uses entity and field names, not tables and columns
- Queries are validated at startup (JPQL only)
- `@Modifying` bypasses the persistence context; set `clearAutomatically`
- Record projections are matched by constructor parameter names

JPQL, NATIVE AND PROJECTIONS

```
interface OrderRepository extends JpaRepository<Order, Long> {

    @Query("""
        select o from Order o
        where o.customer.email = :email and o.total > :min""")
    List<Order> bigOrders(String email, BigDecimal min);

    @Query(value = "select * from orders where created_at > :since",
           nativeQuery = true)
    List<Order> since(Instant since);

    @Modifying(clearAutomatically = true)
    @Transactional
    @Query("update Order o set o.status = :to where o.status = :from")
    int moveAll(Status from, Status to);                // rows updated

    // interface projection
    interface OrderSummary { Long getId(); BigDecimal getTotal(); }
    List<OrderSummary> findByStatus(Status s);

    // record (DTO) projection
    record CustomerTotal(String email, BigDecimal total) { }
    @Query("select new com.example.CustomerTotal(c.email, sum(o.total)) " +
           "from Order o join o.customer c group by c.email")
    List<CustomerTotal> totals();
}
```

CODE EXPLAINED

→ Named params bind by parameter name (`-parameters` flag)

→ `@Modifying` methods return affected-row count

→ Interface projection: only `id`, `total` are selected

→ `new …CustomerTotal(…)` : JPQL constructor expression

JPQL BUILDING BLOCKS

CLAUSE	EXAMPLE
`select`	`select o from Order o`
`join`	`join o.customer c`
`join fetch`	Load relation now
`where`	`o.total > :min`

PARAMETERS

SYNTAX	BINDS
`:name`	By name
`?1`	By position
`:#{#c.id}`	SpEL
`Collection`	`in :ids`

PROJECTION TYPES

TYPE	FETCHES
`Interface`	Getters only
`Record / DTO`	Constructor args
`Dynamic <T>`	`Class<T>` type param
`Entity`	Everything

⚠ COMMON MISTAKES

- Table/column names in JPQL; it expects entity/field names
- `@Modifying` without a transaction → `TransactionRequiredException`
- Native queries tied to one database, then switching to H2 for tests

✓ TIPS & BEST PRACTICES

- Use text blocks (`"""…"""`) for long queries
- Return projections for list screens; load entities only to change them
- Name the parameters exactly like the `:placeholders`

QUICK SUMMARY

- `@Query("jpql")` : custom query
- `nativeQuery = true` : raw SQL
- `@Modifying` : update / delete
- Projections: fetch less

Paging & Sorting

PAGING THROUGH RESULTS

Pass a **Pageable** to a repository method and get a **Page<T>** (content + total count), a **Slice<T>** (content + has-next, no count) or a **Window<T>** for **keyset scrolling**. Build one with `PageRequest.of(page, size, sort)`. In controllers, Spring binds `?page=&size=&sort=` straight into a `Pageable` parameter.

NOTES

- Pages are **zero-based**: `page=0` is the first page
- `Page` runs an extra `count(*)` query; `Slice` doesn't
- Keyset scrolling stays fast on deep pages
- Cap page size with `spring.data.web.pageable.max-page-size`

REPOSITORY AND CONTROLLER

```
interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findByCategory(String category, Pageable pageable);
    Slice<Product> findByActiveTrue(Pageable pageable);
    Window<Product> findTop20ByOrderByIdAsc(ScrollPosition position);
}

@GetMapping("/products")                    // ?page=2&size=20&sort=price,desc
PagedModel<Product> list(@RequestParam String category,
        @PageableDefault(size = 20, sort = "name") Pageable pageable) {
    return new PagedModel<>(repo.findByCategory(category, pageable));
}

// building one yourself
Pageable p = PageRequest.of(0, 50, Sort.by("price").descending()
                                    .and(Sort.by("name")));

// keyset scrolling
Window<Product> w = repo.findTop20ByOrderByIdAsc(ScrollPosition.keyset());
while (!w.isEmpty()) {
    w.forEach(this::export);
    if (!w.hasNext()) break;
    w = repo.findTop20ByOrderByIdAsc(w.positionAt(w.size() - 1));
}
```

CODE EXPLAINED

- → `@PageableDefault` applies when the request has no params
- → `PagedModel` gives stable JSON instead of serializing `PageImpl`
- → `Sort.by(…).descending().and(…)` chains sort orders
- → `positionAt()` remembers where the last window ended

RESULT TYPES

TYPE	GIVES
`Page<T>`	Content + totals
`Slice<T>`	Content + hasNext
`Window<T>`	Keyset / offset scroll
`List<T>`	Just content

PAGE METHODS

METHOD	RETURNS
`getContent()`	`List<T>`
`getTotalElements()`	Row count
`getTotalPages()`	Page count
`hasNext()`	`boolean`
`map(fn)`	Page<R>

REQUEST PARAMS

PARAM	EXAMPLE
`page`	`page=0`
`size`	`size=20`
`sort`	`sort=price,desc`
`sort (repeat)`	`&sort=name`

⚠ COMMON MISTAKES

- Assuming `page=1` is the first page; it's the second
- Returning `Page<Entity>` directly; JSON shape and lazy fields cause trouble
- Deep offset pages on huge tables; use keyset scrolling

✓ TIPS & BEST PRACTICES

- Use `Slice` for infinite scroll; skip the count query
- Set `spring.data.web.pageable.one-indexed-parameters=true` for 1-based APIs
- Map pages to DTOs: `page.map(ProductView::from)`

QUICK SUMMARY

- `Pageable` param → `Page<T>`
- `PageRequest.of(page, size, sort)`
- `Slice`: no count query
- `Window` + `ScrollPosition`: keyset

Transactions

TRANSACTIONS

`@Transactional` on a public bean method runs it in a database transaction: it **commits** when the method returns and **rolls back** on a `RuntimeException`. It works through an AOP proxy, so it only applies to calls **from another bean**. Put it on service methods that change data; use `readOnly = true` for queries.

NOTES

- Checked exceptions **don't** roll back by default
- Default propagation `REQUIRED` joins an existing transaction
- Inside a transaction, loaded entities are **auto-saved** on commit
- Spring's `@Transactional` has more options than Jakarta's

DECLARATIVE AND PROGRAMMATIC

```
@Service
@Transactional(readOnly = true)                  // default for the class
class TransferService {

    @Transactional                               // read-write, overrides
    public void transfer(long from, long to, BigDecimal amount) {
        Account a = accounts.findById(from).orElseThrow();
        Account b = accounts.findById(to).orElseThrow();
        a.withdraw(amount);                      // dirty checking:
        b.deposit(amount);                       // no save() needed
    }                                            // commit here

    @Transactional(rollbackFor = IOException.class, timeout = 10)
    public void importFile(Path p) throws IOException { … }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void audit(String msg) { … }          // own tx, commits alone

    public Balance balance(long id) { … }        // read-only from class
}

// programmatic
transactionTemplate.executeWithoutResult(status -> {
    repo.save(order);
    if (!stock.reserve(order)) status.setRollbackOnly();
});
```

CODE EXPLAINED

- → A `RuntimeException` from `withdraw` rolls back both changes
- → `rollbackFor` adds checked exceptions to the rollback rule
- → `REQUIRES_NEW` suspends the outer transaction
- → `TransactionTemplate` gives fine-grained control in code

PROPAGATION

VALUE	BEHAVIOR
`REQUIRED`	Join or start (default)
`REQUIRES_NEW`	Always a new tx
`SUPPORTS`	Join if present
`MANDATORY`	Error if none

@TRANSACTIONAL ATTRIBUTES

ATTRIBUTE	SETS
`readOnly`	Optimize reads
`rollbackFor`	Extra rollback types
`noRollbackFor`	Commit anyway
`timeout`	Seconds

WHY IT DIDN'T WORK

CAUSE	FIX
`this.method()`	Call via another bean
`private method`	Make it public
`Checked exception`	`rollbackFor`
`Caught exception`	Rethrow it

⚠ COMMON MISTAKES

- Catching the exception inside the method; the transaction commits
- `@Transactional` on controllers; keep transactions in the service layer
- Calling a remote API inside a transaction, holding a DB connection

✓ TIPS & BEST PRACTICES

- Class-level `readOnly = true`, method-level `@Transactional` for writes
- Set the `…transaction` logger to `DEBUG` to see tx boundaries
- Use `@TransactionalEventListener` for work after commit (Sheet 60)

QUICK SUMMARY

- `@Transactional`: commit / rollback
- Runtime exceptions roll back
- `readOnly = true`: queries
- Proxy-based: no self-calls

Fetching & N+1 Problems

LOADING DATA EFFICIENTLY

Lazy relations load on first access, which triggers the **N+1 problem**: one query for the list plus one per row. Accessing them **after** the transaction ends throws **`LazyInitializationException`**. Fix both by fetching what you need up front with **`JOIN FETCH`**, an **`@EntityGraph`**, or a projection, and watch the SQL Hibernate generates.

NOTES

- Boot's `open-in-view` is **on** by default and hides lazy errors
- Set `spring.jpa.open-in-view=false` to catch them early
- Batch fetching loads lazy collections in groups
- Count queries with `show-sql` or Hibernate statistics

FIXING N+1

```
// ❌ N+1: 1 query for orders + 1 per order for its customer
repo.findAll().forEach(o -> System.out.println(o.getCustomer().getName()));

interface OrderRepository extends JpaRepository<Order, Long> {

    @Query("select o from Order o join fetch o.customer where o.status = :s")
    List<Order> withCustomer(Status s);                 // ✅ one query

    @EntityGraph(attributePaths = {"customer", "lines"})
    List<Order> findByStatus(Status s);                 // ✅ graph fetch

    @EntityGraph(attributePaths = "lines")
    Optional<Order> findWithLinesById(long id);
}
```

CODE EXPLAINED

- → `join fetch` loads orders and customers in a single SELECT
- → `@EntityGraph` does the same without writing JPQL
- → Fetching two collections at once causes a cartesian product
- → `default_batch_fetch_size` turns N queries into N/50

DIAGNOSE AND TUNE

```
spring.jpa:
  open-in-view: false
  show-sql: true                          # print SQL (dev only)
  properties.hibernate:
    format_sql: true
    default_batch_fetch_size: 50          # IN (…) for lazy loads
    generate_statistics: true
```

FETCHING OPTIONS

OPTION	WHEN
`JOIN FETCH`	Custom JPQL
`@EntityGraph`	Derived queries
`Projection / DTO`	Read-only lists
`Batch size`	Global safety net

SYMPTOMS

SYMPTOM	CAUSE
`Many similar SELECTs`	N+1
`LazyInitialization…`	Tx closed
`Duplicate parents`	Fetch join list
`MultipleBagFetch…`	2 List fetches

LOGGING SQL

LOGGER	SHOWS
`org.hibernate.SQL`	Statements
`…orm.jdbc.bind`	Bound values
`…stat`	Statistics

⚠ COMMON MISTAKES

- Switching relations to EAGER to silence `LazyInitializationException`
- Fetch-joining two `List` collections → `MultipleBagFetchException`
- Leaving `open-in-view` on so views fire queries during rendering

✓ TIPS & BEST PRACTICES

- Assert query counts in tests (e.g. Hibernate statistics)
- Use `Set` for one of two collections you must fetch together
- Fetch for the use case: different repository methods per screen

QUICK SUMMARY

- N+1: one query per row
- `join fetch` / `@EntityGraph`
- `open-in-view=false`
- `default_batch_fetch_size`

Auditing & Specifications

AUDITING AND DYNAMIC QUERIES

Auditing records who created or changed an entity and when: annotate fields with **`@CreatedDate`**, **`@CreatedBy`** and friends, then turn it on with **`@EnableJpaAuditing`**. **Specifications** build WHERE clauses at runtime from optional filters, ideal for search screens.

NOTES

- `AuditorAware` supplies the current user (e.g. from Spring Security)
- A `@MappedSuperclass` base entity shares audit fields
- Repository must extend `JpaSpecificationExecutor<T>`
- Query by Example: `repo.findAll(Example.of(probe))`

AUDITING

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
abstract class Audited {
    @CreatedDate        Instant createdAt;
    @LastModifiedDate   Instant updatedAt;
    @CreatedBy          String createdBy;
}

@Configuration @EnableJpaAuditing
class AuditConfig {
    @Bean AuditorAware<String> auditor() {      // current user
        return () -> Optional.of(currentUser.name());
    }
}
```

CODE EXPLAINED

→ Auditing fields are set on insert and update automatically

→ `AuditorAware` returns the username, or empty for system jobs

→ A spec returning `null` is ignored, so filters are optional

→ `and()` / `or()` combine specs; paging still works

SPECIFICATIONS

```
static Specification<Product> nameLike(String q) {
    return (root, query, cb) -> q == null ? null
        : cb.like(cb.lower(root.get("name")), "%" + q.toLowerCase() + "%");
}
static Specification<Product> maxPrice(BigDecimal max) {
    return (root, query, cb) -> max == null ? null : cb.le(root.get("price"), max);
}

repo.findAll(nameLike(q).and(maxPrice(max)), pageable);
```

AUDIT ANNOTATIONS

ANNOTATION	SET ON
`@CreatedDate`	Insert
`@LastModifiedDate`	Insert + update
`@CreatedBy`	Insert (user)
`@LastModifiedBy`	Update (user)

CRITERIABUILDER

METHOD	SQL
`cb.equal`	=
`cb.like`	LIKE
`cb.le / ge`	<= / >=

SPECIFICATION API

METHOD	DOES
`where(spec)`	Start a chain
`and / or`	Combine
`not(spec)`	Negate

⚠ COMMON MISTAKES

- Forgetting the `AuditingEntityListener`; audit fields stay `null`
- Using field-name strings that drift from the entity; use the JPA metamodel
- Specs that join collections without `query.distinct(true)`

✓ TIPS & BEST PRACTICES

- Generate the metamodel (`Product_.price`) for type-safe specs
- Use Querydsl if you prefer a fluent, typed query API
- Inject `Clock` via a `DateTimeProvider` to test audit times

QUICK SUMMARY

- `@EnableJpaAuditing`: turn on
- `@CreatedDate` / `@CreatedBy`
- `Specification`: dynamic `WHERE`
- `JpaSpecificationExecutor`: run them

Security Architecture

HOW SPRING SECURITY WORKS

Adding **`spring-boot-starter-security`** secures every endpoint immediately: a login form, HTTP Basic, a generated password in the log, CSRF protection and secure headers. Under the hood a servlet filter (`DelegatingFilterProxy`) hands each request to a **`SecurityFilterChain`**: an ordered list of filters that **authenticate** the caller and then **authorize** the request.

NOTES

- Default user: `user`, password printed at startup
- The result of authentication is an `Authentication` object
- It's stored in the `SecurityContextHolder` (thread-local)
- Boot 4 ships **Spring Security 7** (lambda DSL only)

READING THE CURRENT USER

```
@RestController
class MeController {

    @GetMapping("/me")
    Map<String, Object> me(@AuthenticationPrincipal UserDetails user) {
        return Map.of("name", user.getUsername(),
                      "roles", user.getAuthorities());
    }

    @GetMapping("/whoami")
    String whoami(Authentication auth) {          // injected directly
        return auth.getName();
    }
}

// anywhere in the same thread (e.g. a service)
var auth = SecurityContextHolder.getContext().getAuthentication();
```

CODE EXPLAINED

- → `@AuthenticationPrincipal` injects the logged-in principal
- → An `Authentication` parameter works in any controller method
- → Roles are authorities with a `ROLE_` prefix
- → Fixed dev credentials replace the generated password

QUICK DEV CREDENTIALS

```
spring.security.user.name=admin
spring.security.user.password=change-me
```

KEY FILTERS (IN ORDER)

FILTER	JOB
`SecurityContextHolder…`	Load context
`CsrfFilter`	Check CSRF token
`…AuthenticationFilter`	Log in (form, JWT…)
`ExceptionTranslation…`	401 / 403 / redirect
`AuthorizationFilter`	Allow or deny

CORE OBJECTS

TYPE	HOLDS
`Authentication`	Principal + authorities
`GrantedAuthority`	One permission
`UserDetails`	Username, password, roles
`SecurityContext`	Current Authentication

FAILURE RESPONSES

CASE	RESULT
`Not logged in (API)`	401
`Not logged in (browser)`	Redirect to /login
`Logged in, no access`	403
`Bad CSRF token`	403

⚠ COMMON MISTAKES

- Shipping with the generated password or `spring.security.user.*`
- Reading `SecurityContextHolder` in another thread; it's thread-local
- Assuming security is off for some URLs; it locks down everything

✓ TIPS & BEST PRACTICES

- Log `org.springframework.security` at `DEBUG` to see each filter decision
- Use `@AuthenticationPrincipal` with your own user type
- Read the filter list Boot logs at startup with `DEBUG`

QUICK SUMMARY

- `SecurityFilterChain`: ordered filters
- `Authentication`: who is calling
- `SecurityContextHolder`: where it lives
- 401 = who are you · 403 = not allowed

Configuring HttpSecurity

@EnableWebSecurity · http.authorizeHttpRequests() · requestMatchers() · permitAll() · authenticated() · hasRole() · lambda DSL

DECLARING THE RULES

Customize security with a **`SecurityFilterChain`** bean built from **`HttpSecurity`**. Configure each feature with a lambda: **`authorizeHttpRequests`** lists URL rules in order (first match wins), then pick login mechanisms such as `httpBasic`, `formLogin` or `oauth2ResourceServer`. Defining this bean replaces Boot's default chain.

NOTES

- Rules are checked **top to bottom**; put specific ones first
- End with `anyRequest()` so nothing slips through
- Several chains: use `securityMatcher` + `@Order`
- `.and()` chaining was removed in Security 7; use lambdas

A TYPICAL CONFIGURATION

```
@Configuration
@EnableWebSecurity
class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/css/**", "/actuator/health").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults())
            .formLogin(form -> form.loginPage("/login").permitAll())
            .csrf(csrf -> csrf.ignoringRequestMatchers("/api/**"))
            .cors(Customizer.withDefaults());
        return http.build();
    }
}
```

CODE EXPLAINED

- → `permitAll()` lets anonymous users through
- → `hasRole("ADMIN")` checks for authority `ROLE_ADMIN`
- → `Customizer.withDefaults()` enables a feature with defaults
- → CSRF is skipped for the stateless `/api/**` endpoints

AUTHORIZATION RULES

METHOD	ALLOWS
`permitAll()`	Everyone
`authenticated()`	Logged-in users
`hasRole("X")`	ROLE_X
`hasAuthority("x")`	Exact authority
`denyAll()`	Nobody

REQUESTMATCHERS

FORM	MATCHES
`"/a/**"`	Path and below
`"/a/*"`	One segment
`HttpMethod.POST, …`	Method + path
`PathRequest…`	Static resources

HTTPSECURITY FEATURES

METHOD	CONFIGURES
`httpBasic`	Basic auth
`formLogin`	Login page
`oauth2Login`	Social / OIDC
`sessionManagement`	Sessions
`headers`	Security headers

⚠ COMMON MISTAKES

- `anyRequest()` before specific rules; later matchers never run
- `hasRole("ROLE_ADMIN")`; the prefix is added for you
- Disabling CSRF for a browser app that uses session cookies

✓ TIPS & BEST PRACTICES

- Permit CSS/JS/images with `PathRequest.toStaticResources()`
- Prefer method security for business rules (Sheet 48)
- Write a `@WebMvcTest` per rule with `@WithMockUser` (Sheet 55)

QUICK SUMMARY

- `SecurityFilterChain` bean: your rules
- `authorizeHttpRequests`: first match
- `hasRole` / `permitAll` / `authenticated`
- Lambda DSL only in Security 7

Users & Password Encoding

WHERE USERS COME FROM

For username/password login, Spring Security asks a **`UserDetailsService`** to load a user by name, then checks the password with a **`PasswordEncoder`**. Use **`InMemoryUserDetailsManager`** for demos, `JdbcUserDetailsManager` for the standard schema, or implement the interface over your own user table. Passwords are always stored **hashed**, never in plain text.

NOTES

- Default encoder is **delegating**: hashes start with `{bcrypt}`
- `{id}` prefix lets old and new algorithms coexist
- `User.withUsername(…)` builds `UserDetails` objects
- Expose one `UserDetailsService` bean; Boot wires the rest

USERS FROM YOUR DATABASE

```
@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

@Service
class AppUserDetailsService implements UserDetailsService {
    private final AppUserRepository users;
    AppUserDetailsService(AppUserRepository users) { this.users = users; }

    @Override
    public UserDetails loadUserByUsername(String email) {
        AppUser u = users.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException(email));
        return User.withUsername(u.email())
            .password(u.passwordHash())            // "{bcrypt}$2a$10$..."
            .roles(u.role().name())                // -> ROLE_USER
            .disabled(!u.active())
            .build();
    }
}
```

CODE EXPLAINED

- → `UsernameNotFoundException` makes login fail with a generic error
- → `.roles("USER")` adds authority `ROLE_USER`
- → `encode()` produces a new salted hash each call
- → Use `matches(raw, hash)` to check a password yourself

ENCODERS

ID	ALGORITHM
`{bcrypt}`	BCrypt (default)
`{argon2}`	Argon2
`{pbkdf2}`	PBKDF2
`{scrypt}`	SCrypt
`{noop}`	Plain text (tests)

USER BUILDER

METHOD	SETS
`withUsername`	Login name
`password`	Encoded hash
`roles`	ROLE_* authorities
`authorities`	Raw authorities
`disabled / accountLocked`	Status flags

BUILT-IN SERVICES

CLASS	STORES IN
`InMemoryUserDetails…`	Memory
`JdbcUserDetails…`	Standard tables
`Your implementation`	Any repository

⚠ COMMON MISTAKES

- Storing passwords with `{noop}` or a plain `MessageDigest` hash
- Comparing hashes with `equals`; use `encoder.matches()`
- Telling users whether the username or the password was wrong

✓ TIPS & BEST PRACTICES

- Keep the delegating encoder so you can upgrade algorithms later
- Use a `UserDetailsPasswordService` to re-hash on login
- Return your own `UserDetails` type to carry extra fields like an id

QUICK SUMMARY

- `UserDetailsService`: load by name
- `PasswordEncoder`: hash + match
- `{bcrypt}`: delegating default
- `User.withUsername()`: build users

Method Security

SECURING METHODS

URL rules guard endpoints; **method security** guards business logic wherever it's called. Enable it with **`@EnableMethodSecurity`**, then annotate service methods with **`@PreAuthorize`** (check before running) or **`@PostAuthorize`** (check the return value). Expressions are **SpEL** and can use the method's arguments, the current `authentication` and your own beans.

NOTES

- Works through AOP proxies, like `@Transactional`
- Denied calls throw `AuthorizationDeniedException` → 403
- `@Secured` / `@RolesAllowed` are simpler, role-only options
- `@PostFilter` removes items the user may not see

RULES ON SERVICE METHODS

```
@Configuration
@EnableMethodSecurity
class MethodSecurityConfig { }

@Service
class DocumentService {

    @PreAuthorize("hasRole('ADMIN')")
    public void purge() { … }

    @PreAuthorize("#doc.ownerId == authentication.name")
    public void update(Document doc) { … }

    @PostAuthorize("returnObject.ownerId == authentication.name")
    public Document read(long id) { return repo.findById(id).orElseThrow(); }

    @PreAuthorize("@acl.canEdit(#id, authentication)")    // custom bean
    public void rename(long id, String name) { … }

    @PostFilter("filterObject.ownerId == authentication.name")
    public List<Document> mine() { return repo.findAll(); }
}
```

CODE EXPLAINED

- → `#doc` refers to the method parameter named `doc`
- → `returnObject` is the value the method returned
- → `@acl` calls a bean named `acl` (your own `@Component`)
- → `filterObject` is each element of the returned list

ANNOTATIONS

ANNOTATION	CHECKS
`@PreAuthorize`	Before the call
`@PostAuthorize`	After, on result
`@PreFilter`	Filter arguments
`@PostFilter`	Filter results
`@Secured`	Roles only

SPEL HELPERS

EXPRESSION	TRUE IF
`hasRole('X')`	Has ROLE_X
`hasAuthority('x')`	Has authority
`isAuthenticated()`	Logged in
`authentication.name`	Username
`#param`	Method argument

URL VS METHOD RULES

ASPECT	BEST FOR
`URL rules`	Coarse access
`@PreAuthorize`	Business rules
`@PostAuthorize`	Ownership checks

⚠ COMMON MISTAKES

- Forgetting `@EnableMethodSecurity`; annotations are silently ignored
- Calling a secured method from the same class; the proxy is bypassed
- `@PostAuthorize` on methods with side effects; they already ran

✓ TIPS & BEST PRACTICES

- Create meta-annotations like `@IsAdmin` for repeated expressions
- Move complex checks into a bean and call it with `@bean.method()`
- Test with `@WithMockUser(roles = "ADMIN")` (Sheet 55)

QUICK SUMMARY

- `@EnableMethodSecurity`: turn on
- `@PreAuthorize("…")`: check first
- `@PostAuthorize`: check result
- `#arg`, `authentication`, `@bean`

Form Login, Sessions & CSRF

BROWSER SESSIONS AND CSRF

Browser apps usually use **form login** with an HTTP **session**: after login the `JSESSIONID` cookie identifies the user. Because the browser sends cookies automatically, these apps need **CSRF protection** (on by default): every state-changing request must carry a token. Stateless token APIs instead set the session policy to **STATELESS** and can disable CSRF.

NOTES

- Thymeleaf forms include the CSRF token automatically
- SPAs read the token from an `XSRF-TOKEN` cookie
- `rememberMe()` keeps users logged in across browser restarts
- Login changes the session id (session fixation protection)

SESSION-BASED WEB APP

```
@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(a -> a
            .requestMatchers("/login", "/css/**").permitAll()
            .anyRequest().authenticated())
        .formLogin(f -> f
            .loginPage("/login")
            .defaultSuccessUrl("/dashboard"))
        .logout(l -> l.logoutSuccessUrl("/login?logout"))
        .rememberMe(r -> r.tokenValiditySeconds(14 * 24 * 3600))
        .sessionManagement(s -> s.maximumSessions(1))
        .csrf(c -> c.spa());                    // cookie-based token for SPAs
    return http.build();
}
```

CODE EXPLAINED

- → Custom `/login` page must post `username` and `password`
- → `logout` is a POST by default (needs the CSRF token)
- → `maximumSessions(1)` limits each user to one login
- → `csrf.spa()` (Security 6.4+) sets up cookie + header for JS apps

STATELESS API

```
http.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
    .csrf(c -> c.disable());     // safe only without cookie auth
```

SESSION POLICIES

POLICY	SESSION
IF_REQUIRED	Create when needed
ALWAYS	Always create
NEVER	Use if exists
STATELESS	Never use

CSRF HANDLING

CLIENT	SENDS TOKEN AS
Thymeleaf form	Hidden `_csrf` field
SPA (JS)	X-XSRF-TOKEN header
Mobile / JWT API	Not needed

SESSION PROPERTIES

PROPERTY	SETS
…session.timeout	Idle timeout
…cookie.secure	HTTPS only
…cookie.same-site	lax / strict
…cookie.http-only	No JS access

⚠ COMMON MISTAKES

- Disabling CSRF in a cookie-session app to make a form work
- Making logout a GET link; Spring expects POST with a token
- Storing large objects in the session; it hurts memory and clustering

✓ TIPS & BEST PRACTICES

- Use Spring Session (Redis/JDBC) to share sessions across instances
- Set `same-site=strict` cookies where possible
- Use a token API (Sheet 50) for mobile clients instead of sessions

QUICK SUMMARY

- `formLogin()` : session login
- CSRF: on by default, keep it for cookies
- `STATELESS` : token APIs
- `rememberMe()` : long-lived login

JWT Resource Server

PROTECTING APIS WITH JWTS

As an **OAuth2 resource server**, your API accepts **bearer tokens** (usually JWTs) issued by an authorization server such as Keycloak, Auth0, Okta or Spring Authorization Server. Set the **`issuer-uri`** and Boot fetches the signing keys, validates each token's signature, expiry and issuer, and turns its **scopes** into authorities.

NOTES

- Add `spring-boot-starter-security-oauth2-resource-server`
- Scopes become `SCOPE_read`, `SCOPE_write` authorities
- The token's `sub` claim becomes `authentication.getName()`
- No sessions: every request carries its own token

RESOURCE SERVER SETUP

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(a -> a
            .requestMatchers(GET, "/api/**").hasAuthority("SCOPE_read")
            .requestMatchers("/api/**").hasAuthority("SCOPE_write")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS));
    return http.build();
}

@GetMapping("/api/me")
Map<String, Object> me(@AuthenticationPrincipal Jwt jwt) {
    return Map.of("sub", jwt.getSubject(),
                  "email", jwt.getClaimAsString("email"));
}
```

CODE EXPLAINED

→ Boot reads `/.well-known/openid-configuration` from the issuer

→ `audiences` rejects tokens meant for other APIs

→ `@AuthenticationPrincipal Jwt` gives access to every claim

→ Map a `roles` claim with a `JwtAuthenticationConverter` bean

APPLICATION.YML

```
spring.security.oauth2.resourceserver.jwt:
  issuer-uri: https://auth.example.com/realms/shop
  audiences: shop-api
```

JWT.* PROPERTIES

PROPERTY	PURPOSE
`issuer-uri`	Discover keys + validate
`jwk-set-uri`	Keys URL only
`audiences`	Expected aud
`public-key-location`	Static RSA key

COMMON CLAIMS

CLAIM	MEANING
`sub`	User id
`iss`	Issuer
`aud`	Intended API
`exp`	Expiry
`scope / scp`	Granted scopes

TOKEN ERRORS

PROBLEM	RESULT
`No token`	401
`Expired / bad sig`	401 + `WWW-Authenticate`
`Missing scope`	403

⚠ COMMON MISTAKES

- Validating JWTs by hand instead of using the resource server support
- Putting sensitive data in JWT claims; anyone can decode them
- Accepting any audience, so tokens for other apps work on yours

✓ TIPS & BEST PRACTICES

- Use opaque tokens (`opaquetoken`) when you need instant revocation
- Test with `jwt()` from `spring-security-test` (Sheet 55)
- Run Keycloak in Docker Compose for local development

QUICK SUMMARY

- `oauth2ResourceServer().jwt()`
- `issuer-uri` : keys + validation
- Scopes → `SCOPE_*` authorities
- `@AuthenticationPrincipal Jwt`

OAuth2 Login & Client

LOGGING IN WITH OAUTH2 / OIDC

`oauth2Login()` lets users sign in through an external provider (Google, GitHub, Okta, Keycloak) using **OpenID Connect**. Register the provider under `spring.security.oauth2.client.registration`, and Spring handles the redirect, code exchange and user info. As an OAuth2 **client**, your app can also call other APIs with the user's access token.

NOTES

- Add `spring-boot-starter-security-oauth2-client`
- Google, GitHub, Facebook and Okta have built-in provider defaults
- Callback path: `/login/oauth2/code/{id}`
- OIDC users arrive as `OidcUser`; plain OAuth2 as `OAuth2User`

CONFIGURATION

```yaml
spring.security.oauth2.client:
  registration:
    google:
      client-id: ${GOOGLE_CLIENT_ID}
      client-secret: ${GOOGLE_CLIENT_SECRET}
      scope: openid, profile, email
    keycloak:
      client-id: shop-web
      client-secret: ${KC_SECRET}
      scope: openid
  provider:
    keycloak:
      issuer-uri: https://auth.example.com/realms/shop
```

CODE EXPLAINED

→ Google needs only id, secret and scopes

→ Custom providers need an `issuer-uri` for discovery

→ Visiting a protected page redirects to the provider

→ `OidcUser` exposes the ID token's claims

SECURITY + CONTROLLER

```java
http.authorizeHttpRequests(a -> a.anyRequest().authenticated())
    .oauth2Login(Customizer.withDefaults())
    .logout(l -> l.logoutSuccessUrl("/"));

@GetMapping("/profile")
String profile(@AuthenticationPrincipal OidcUser user, Model model) {
    model.addAttribute("name", user.getFullName());
    return "profile";
}
```

LOGIN FLOW

STEP	WHAT HAPPENS
1	Redirect to provider
2	User signs in
3	Back with code
4	Code → tokens

CALLING APIS AS THE USER

TOOL	USE
`@RegisteredOAuth2…`	Inject client in MVC
`OAuth2AuthorizedClient…`	Manager bean
`RestClient interceptor`	Adds bearer token

REGISTRATION KEYS

KEY	VALUE
`client-id`	From provider
`client-secret`	From env var
`scope`	`openid, profile`
`redirect-uri`	Default is fine

⚠ COMMON MISTAKES

- Committing `client-secret` to the repo; use env vars
- Redirect URI in the provider console not matching the app's
- Using the ID token to call APIs; use the access token

✓ TIPS & BEST PRACTICES

- Map provider claims to local users in a custom `OidcUserService`
- Use `OAuth2ClientHttpRequestInterceptor` with `RestClient` for downstream calls
- For SPAs, consider a backend-for-frontend with this setup

QUICK SUMMARY

- `oauth2Login()`: social / OIDC login
- `registration.<id>`: client settings
- `OidcUser`: the signed-in user
- Access token: for API calls

@SpringBootTest

FULL INTEGRATION TESTS

`@SpringBootTest` starts the whole application context for a JUnit test, just like `main()` would, so you can inject real beans. By default it uses a mock servlet environment; with **`webEnvironment = RANDOM_PORT`** it starts a real server. Spring **caches** contexts between test classes with the same configuration, so keep that configuration consistent.

NOTES

- Needs `spring-boot-starter-test` (JUnit, AssertJ, Mockito)
- Boot 4: MockMvc and test clients need `@AutoConfigure…` annotations
- `@ActiveProfiles` picks profiles for the test
- `@DynamicPropertySource` sets properties from code

INTEGRATION TEST

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
@ActiveProfiles("test")
class OrderFlowTests {

    @Autowired RestTestClient client;          // bound to the running server
    @Autowired OrderRepository orders;

    @Test
    void placesAnOrder() {
        client.post().uri("/api/orders")
            .body(new NewOrder("book", 2))
            .exchange()
            .expectStatus().isCreated();

        assertThat(orders.count()).isEqualTo(1);
    }

    @TestConfiguration                          // extra beans for this test
    static class Clocks {
        @Bean Clock clock() { return Clock.fixed(Instant.EPOCH, ZoneOffset.UTC); }
    }
}
```

CODE EXPLAINED

- → `RANDOM_PORT` avoids clashes when tests run in parallel
- → Test classes can use field injection with `@Autowired`
- → `@TestConfiguration` adds beans without replacing the app config
- → Different properties or mocks = a new (slower) context

WEBENVIRONMENT

VALUE	SERVER
`MOCK (default)`	Mock servlet
`RANDOM_PORT`	Real, random port
`DEFINED_PORT`	Real, `server.port`
`NONE`	No web at all

USEFUL ANNOTATIONS

ANNOTATION	DOES
`@ActiveProfiles`	Set profiles
`@TestPropertySource`	Override props
`@DirtiesContext`	Discard context
`@Sql`	Run SQL scripts
`@Transactional`	Roll back each test

@AUTOCONFIGURE…

SUFFIX	INJECTS
`MockMvc`	`MockMvcTester`
`RestTestClient`	`RestTestClient`
`TestRestTemplate`	`TestRestTemplate`

⚠ COMMON MISTAKES

- `@SpringBootTest` for everything; slices are much faster (Sheet 53)
- `@DirtiesContext` everywhere; it throws away the cached context
- `@Transactional` on `RANDOM_PORT` tests; the server runs another thread

✓ TIPS & BEST PRACTICES

- Share one base test class so contexts are cached and reused
- Use Testcontainers for the database (Sheet 56)
- Inject `Clock` so time-based logic is testable

QUICK SUMMARY

- `@SpringBootTest`: whole app
- `RANDOM_PORT`: real server
- `@TestConfiguration`: extra beans
- Context cache: keep config stable

Test Slices

TESTING ONE LAYER

Test slices start only the part of the app you're testing, so they're much faster than `@SpringBootTest`. **`@WebMvcTest`** loads controllers, advice, filters and JSON, but not services; **`@DataJpaTest`** loads JPA, repositories and an embedded database and rolls back each test. Mock what's outside the slice with `@MockitoBean`.

NOTES

- Boot 4: slices live in tech-specific test modules (`-test` starters)
- `@WebMvcTest(Controller.class)` limits it to one controller
- `@DataJpaTest` replaces your DataSource with an embedded DB

WEB SLICE

```
@WebMvcTest(OrderController.class)
class OrderControllerTests {

    @Autowired MockMvcTester mvc;
    @MockitoBean OrderService service;          // not in the slice → mock

    @Test void returnsOrder() {
        given(service.find(7)).willReturn(new Order(7, "book"));

        assertThat(mvc.get().uri("/api/orders/7"))
            .hasStatusOk()
            .bodyJson().extractingPath("$.item").isEqualTo("book");
    }
}
```

CODE EXPLAINED

→ Only `OrderController` and MVC infrastructure are loaded

→ `given(...)` is BDD-style Mockito stubbing

→ `TestEntityManager` can also insert rows directly

DATA SLICE

```
@DataJpaTest
class OrderRepositoryTests {
    @Autowired OrderRepository repo;

    @Test void findsByStatus() {
        repo.save(new Order("book", Status.NEW));
        assertThat(repo.findByStatus(Status.NEW)).hasSize(1);
    }                                           // rolled back afterwards
}
```

COMMON SLICES

ANNOTATION	LOADS
`@WebMvcTest`	Controllers, MVC
`@DataJpaTest`	JPA, repositories
`@JdbcTest`	JDBC, JdbcClient
`@JsonTest`	Jackson mappers

@WEBMVCTEST EXCLUDES

NOT LOADED	SO…
`@Service`	`@MockitoBean` it
`@Repository`	Mock it
`@Configuration`	`@Import` it

SLICE EXTRAS

ANNOTATION	ADDS
`@Import(X.class)`	Specific beans
`@AutoConfigure…`	Extra auto-config
`@TestPropertySource`	Properties

⚠ COMMON MISTAKES

- Importing half the app into a slice; use `@SpringBootTest` instead
- Expecting security to be off in `@WebMvcTest`; it's on (Sheet 55)
- Trusting the embedded DB; keep the real one with `replace = NONE`

✓ TIPS & BEST PRACTICES

- Test controllers with slices, flows with one `@SpringBootTest`
- Use `@JsonTest` + `JacksonTester` for tricky DTO serialization
- Check which slice annotations your `-test` starter provides

QUICK SUMMARY

- `@WebMvcTest`: controller layer
- `@DataJpaTest`: repository layer
- `@MockitoBean`: fill gaps
- Slices: fast, focused tests

MockMvc & RestTestClient

TESTING HTTP ENDPOINTS

`MockMvcTester` (Framework 6.2+) tests Spring MVC without a server, using **AssertJ**: `assertThat(mvc.get().uri("/x")).hasStatusOk()`. **`RestTestClient`** (Framework 7) offers a fluent `expectStatus()` / `expectBody()` API and can target MockMvc, a single controller or a **real running server**. Both check status, headers and JSON bodies.

NOTES

- `MockMvcTester` replaces the older `MockMvc` + `ResultMatchers` style
- `bodyJson()` supports JSON paths and comparisons
- `RestTestClient.bindToController()` needs no Spring context
- `.body(obj)` serializes with Jackson

MOCKMVCTESTER (ASSERTJ)

```
assertThat(mvc.post().uri("/api/orders")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"item\":\"book\",\"qty\":2}"))
    .hasStatus(HttpStatus.CREATED)
    .hasHeader("Location", "/api/orders/1")
    .bodyJson().extractingPath("$.qty").isEqualTo(2);

assertThat(mvc.get().uri("/api/orders/99"))
    .hasStatus(HttpStatus.NOT_FOUND)
    .bodyJson().extractingPath("$.title").isEqualTo("Order not found");
```

RESTTESTCLIENT

```
RestTestClient client = RestTestClient
    .bindToController(new OrderController(fakeService)).build();

client.get().uri("/api/orders/7")
    .exchange()
    .expectStatus().isOk()
    .expectBody()
    .jsonPath("$.item").isEqualTo("book");

Order o = client.get().uri("/api/orders/7").exchange()
    .expectBody(Order.class).returnResult().getResponseBody();
```

CODE EXPLAINED

→ `mvc` is injected in `@WebMvcTest` or with `@AutoConfigureMockMvc`

→ Error bodies can be checked against `ProblemDetail` fields

→ `bindToController` tests one controller in plain JUnit

→ `returnResult()` exits the chain for custom assertions

MOCKMVCTESTER ASSERTS

METHOD	CHECKS
`hasStatusOk()`	200
`hasStatus(s)`	Any status
`hasHeader(k, v)`	Header value
`bodyJson()`	JSON assertions
`hasFailed()`	Exception thrown

RESTTESTCLIENT BINDS

METHOD	TARGET
`bindToController`	One controller
`bindToApplicationContext`	MVC config
`bindToServer`	Running app
`bindToRouterFunction`	Functional

EXPECTATIONS

METHOD	CHECKS
`expectStatus()`	Status
`expectHeader()`	Headers
`expectBody(T.class)`	Decoded body
`jsonPath(p)`	JSON value
`expectAll(…)`	Soft assertions

⚠ COMMON MISTAKES

- Mixing old `andExpect(status().isOk())` style with AssertJ; pick one
- Testing JSON as an exact string; field order and spacing break it
- Using `bindToServer` without `RANDOM_PORT`; nothing is listening

✓ TIPS & BEST PRACTICES

- `bodyJson().isLenientlyEqualTo(json)` ignores order and extra fields
- Prefer `RestTestClient` in `RANDOM_PORT` tests over `TestRestTemplate`
- Name tests after behavior: `returns404ForUnknownOrder`

QUICK SUMMARY

- `MockMvcTester`: AssertJ MVC tests
- `RestTestClient`: fluent client tests
- `bodyJson()` / `jsonPath()`
- `bindToController`: no context

Mocking Beans

MOCKS AND TEST USERS

@MockitoBean replaces a bean in the test context with a Mockito mock; **@MockitoSpyBean** wraps the real bean so you can verify calls. They replace Boot's removed `@MockBean` / `@SpyBean`. For secured endpoints, **spring-security-test** adds **@WithMockUser** and request post-processors such as `jwt()` and `csrf()`.

NOTES

- Mocks are reset after each test automatically
- Each different set of mocked beans creates a new context
- Boot 4: add `spring-boot-starter-security-test` for `@WithMockUser`
- Plain unit tests: use `@ExtendWith(MockitoExtension.class)` + `@Mock`

MOCKS AND SECURITY IN A SLICE

```
@WebMvcTest(AdminController.class)
class AdminControllerTests {

    @Autowired MockMvcTester mvc;
    @MockitoBean UserService users;            // mock
    @MockitoSpyBean AuditLog audit;            // real bean, spied

    @Test
    @WithMockUser(roles = "ADMIN")
    void adminCanDelete() {
        assertThat(mvc.delete().uri("/admin/users/5").with(csrf()))
            .hasStatus(HttpStatus.NO_CONTENT);
        verify(users).delete(5L);
        verify(audit).record(contains("deleted 5"));
    }

    @Test
    void anonymousIsRejected() {
        assertThat(mvc.delete().uri("/admin/users/5").with(csrf()))
            .hasStatus(HttpStatus.UNAUTHORIZED);
    }
}
```

CODE EXPLAINED

- → `csrf()` adds a valid token to state-changing requests
- → `verify(users).delete(5L)` checks the mock was called
- → Without a user the API returns 401
- → For token APIs: `.with(jwt())` fakes a validated JWT

MOCKITO BASICS

CALL	DOES
`given(x).willReturn(v)`	Stub
`willThrow(e)`	Stub an error
`verify(m).call()`	Assert called
`verify(m, never())`	Assert not called
`ArgumentCaptor`	Capture args

TEST USERS

ANNOTATION / METHOD	GIVES
`@WithMockUser`	User + roles
`@WithUserDetails`	From your service
`@WithAnonymousUser`	Anonymous
`user("ann")`	Per request
`jwt()` / `oidcLogin()`	Token users

BOOT 3 → 4

OLD	NEW
`@MockBean`	`@MockitoBean`
`@SpyBean`	`@MockitoSpyBean`
`in @Configuration`	Class-level `types=`

⚠ COMMON MISTAKES

- Forgetting `.with(csrf())` on POST/PUT/DELETE; you get 403, not your status
- Mocking the class under test; you test Mockito, not your code
- Many unique `@MockitoBean` combos, each spawning a new context

✓ TIPS & BEST PRACTICES

- Put shared mocks on a base class or meta-annotation
- Prefer real objects over mocks for simple value collaborators
- Use `@WithMockUser` on the class to cover every test

QUICK SUMMARY

- `@MockitoBean`: mock a bean
- `@MockitoSpyBean`: spy a real bean
- `@WithMockUser`: fake login
- `csrf()` / `jwt()`: request helpers

Testcontainers

REAL SERVICES IN TESTS

Testcontainers starts real databases and brokers in Docker for your tests. With **`@ServiceConnection`** on a container field, Boot reads its host, port and credentials and configures the matching connection automatically: no `@DynamicPropertySource` needed. The same containers can power local development through a **test main class**.

NOTES

- Needs Docker (or a compatible runtime) on the machine
- Boot 4 uses **Testcontainers 2**: new packages, e.g. `org.testcontainers.postgresql`
- Artifacts: `spring-boot-testcontainers` + `testcontainers-postgresql`
- `static` containers are shared by all tests in the class

CONTAINER PER TEST CLASS

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {

    @Container
    @ServiceConnection                          // sets spring.datasource.*
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17");

    @Autowired OrderRepository repo;      // tests hit real Postgres
}
```

SHARED CONFIG + DEV MODE

```
@TestConfiguration(proxyBeanMethods = false)
class Containers {
    @Bean @ServiceConnection PostgreSQLContainer pg() {
        return new PostgreSQLContainer("postgres:17");
    }
}
class TestShopApplication {                 // dev mode, in src/test
    public static void main(String[] args) {
        SpringApplication.from(ShopApplication::main)
            .with(Containers.class).run(args);
    }
}
```

CODE EXPLAINED

- → `@Testcontainers` starts and stops `@Container` fields
- → Testcontainers 2's `PostgreSQLContainer` has no generic parameter
- → Containers as beans are shared across tests via the context cache
- → `SpringApplication.from(…).with(…)` runs the app plus containers

@SERVICECONNECTION SUPPORTS

CONTAINER	CONFIGURES
`PostgreSQL / MySQL`	DataSource
`MongoDB`	Mongo client
`Redis`	Redis connection
`Kafka`	Kafka bootstrap

MAKING IT FAST

TECHNIQUE	EFFECT
`static container`	One per class
`@Bean container`	One per context
`withReuse(true)`	Keep between runs
`Same image tag`	Cached pulls

RUNNING

COMMAND	RUNS
`./mvnw verify`	Unit + IT tests
`./gradlew test`	All tests
`spring-boot:test-run`	Test main (Maven)
`bootTestRun`	Test main (Gradle)

⚠ COMMON MISTAKES

- Non-static `@Container` fields; a new container starts for every test
- Using `latest` image tags; tests change when the image does
- Keeping H2-only tests for queries that use Postgres features

✓ TIPS & BEST PRACTICES

- Match the production database version in the image tag
- Run Testcontainers ITs in CI with Docker available
- Use the test main class instead of a hand-made Docker Compose for dev

QUICK SUMMARY

- `@Container` + `@ServiceConnection`
- Real DB, auto-configured URL
- Containers as `@Bean`s: shared
- `bootTestRun`: dev with containers

Actuator Endpoints

PRODUCTION ENDPOINTS

Actuator adds operational endpoints under **`/actuator`**: health, info, metrics, environment, loggers, beans and more. Only **`health`** is exposed over HTTP by default; expose others explicitly and secure them. Health combines **indicators** for the database, disk, Redis and so on, and Kubernetes **liveness/readiness** probes come built in.

NOTES

- Add `spring-boot-starter-actuator`
- Run Actuator on a separate port with `management.server.port`
- Health details are hidden unless `show-details` allows them
- `/actuator/loggers` changes log levels at runtime (POST)

APPLICATION.YML

```yaml
management:
  endpoints.web.exposure.include: health, info, metrics, prometheus, loggers
  endpoint.health:
    show-details: when-authorized
    probes.enabled: true          # /actuator/health/liveness, /readiness
  info.env.enabled: true
info.app:
  name: shop
  version: '@project.version@'    # Maven resource filtering
```

CUSTOM HEALTH INDICATOR

```java
@Component
class PaymentGatewayHealth implements HealthIndicator {
    private final PaymentClient client;
    PaymentGatewayHealth(PaymentClient client) { this.client = client; }

    @Override
    public Health health() {
        return client.ping()
            ? Health.up().withDetail("latencyMs", client.lastLatency()).build()
            : Health.down().withDetail("reason", "no response").build();
    }
}
```

CODE EXPLAINED

→ Exposed endpoints appear at `/actuator/{id}`

→ `when-authorized` shows details to users with a role

→ Bean `PaymentGatewayHealth` shows up as `paymentGateway`

→ Any `DOWN` indicator makes overall health `DOWN` (503)

COMMON ENDPOINTS

ENDPOINT	SHOWS
`health`	Up / down
`info`	App info
`metrics`	Micrometer meters
`env`	Properties (masked)
`mappings`	All URL mappings

MORE ENDPOINTS

ENDPOINT	SHOWS
`loggers`	View / set levels
`beans`	All beans
`conditions`	Auto-config report
`threaddump`	Thread stacks
`heapdump`	Heap file

MANAGEMENT.*

PROPERTY	SETS
`server.port`	Separate port
`endpoints.web.base-path`	`/actuator`
`…exposure.include`	Exposed list
`…show-details`	Health detail

⚠ COMMON MISTAKES

- `exposure.include=*` on a public port; `env` and `heapdump` leak secrets
- Probes that check external services; one outage restarts every pod
- Forgetting to secure Actuator when Spring Security is present

✓ TIPS & BEST PRACTICES

- Put Actuator on an internal `management.server.port`
- Group indicators: `management.endpoint.health.group.*`
- Add a `@ReadOperation` custom endpoint with `@Endpoint(id = "…")`

QUICK SUMMARY

- `/actuator/health`: liveness
- `exposure.include`: opt in
- `HealthIndicator`: custom checks
- Separate port + security

Metrics & Tracing

Micrometer · MeterRegistry · Counter · Timer · @Observed · Prometheus endpoint · OpenTelemetry

MEASURING THE APP

Boot instruments the app with **Micrometer**: HTTP requests, JVM, connection pools and caches are measured automatically. Add a registry (e.g. **Prometheus**) to export them. Record your own **counters**, **timers** and **gauges** with a `MeterRegistry`, or annotate code with **@Observed** to get a metric *and* a trace span. Tracing exports to **OpenTelemetry**.

NOTES

- Prometheus: add `micrometer-registry-prometheus`
- Tracing: `spring-boot-starter-opentelemetry` (Boot 4)
- Trace and span ids are added to log lines automatically
- Tag cardinality must stay low: no user ids in tags

CUSTOM METRICS

```
@Service
class CheckoutService {
    private final Counter orders;
    private final Timer payment;

    CheckoutService(MeterRegistry registry) {
        this.orders = Counter.builder("shop.orders")
            .tag("channel", "web").register(registry);
        this.payment = registry.timer("shop.payment.time");
        Gauge.builder("shop.cart.open", carts, CartStore::size).register(registry);
    }

    void checkout(Cart cart) {
        payment.record(() -> gateway.charge(cart));
        orders.increment();
    }

    @Observed(name = "shop.lookup")              // metric + span
    Product lookup(String sku) { … }
}
```

CODE EXPLAINED

→ Meter names use dots; Prometheus shows `shop_orders_total`

→ `timer.record(…)` times the lambda

→ Gauges sample a value when scraped

→ Turn `@Observed` on with `… observations.annotations.enabled`

APPLICATION.YML

```
management:
  endpoints.web.exposure.include: prometheus
  tracing.sampling.probability: 0.1      # trace 10% of requests
```

METER TYPES

TYPE	MEASURES
`Counter`	Count up only
`Timer`	Duration + count
`Gauge`	Current value
`DistributionSummary`	Sizes, amounts

BUILT-IN METRICS

NAME	COVERS
`http.server.requests`	MVC requests
`jvm.memory.used`	Heap / non-heap
`hikaricp.connections`	DB pool
`cache.gets`	Cache hits
`process.cpu.usage`	CPU

INSPECT

URL	SHOWS
`/actuator/metrics`	Meter names
`…/metrics/{name}`	Values + tags
`/actuator/prometheus`	Scrape format

⚠ COMMON MISTAKES

- Tagging with unbounded values (user id, URL with ids); memory explodes
- Sampling 100% of traces in production
- Creating meters inside hot methods instead of once

✓ TIPS & BEST PRACTICES

- Tag all meters with the app name via `… metrics.tags.application`
- Use Grafana dashboards for JVM and HTTP metrics
- Prefer `@Observed` / `Observation` for code you want traced

QUICK SUMMARY

- Micrometer: metrics facade
- `MeterRegistry`: create meters
- `@Observed`: metric + trace
- `/actuator/prometheus`: export

Logging

LOGGING

Boot logs through **SLF4J** with **Logback** by default and prints colored console output. Set levels per package with **`logging.level.*`**, write to a file with `logging.file.name`, and switch to **structured JSON** logs with one property. For full control, add `logback-spring.xml`, which supports `<springProfile>` blocks.

NOTES

- Default level is `INFO`
- Use parameterized messages: `log.info("id={}", id)`
- MDC values (`traceId` , your own keys) appear in every line

LOGGING IN CODE

```
@Service
class OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    void place(Order o) {
        log.debug("Placing order {}", o.id());          // lazy formatting
        MDC.put("orderId", String.valueOf(o.id()));
        try {
            gateway.charge(o);
            log.info("Order {} placed, total={}", o.id(), o.total());
        } catch (PaymentException e) {
            log.error("Payment failed for {}", o.id(), e);   // stack trace
        } finally {
            MDC.remove("orderId");
        }
    }
}
```

CODE EXPLAINED

- → `{}` placeholders skip string building when the level is off
- → The exception as last argument prints its stack trace
- → MDC adds `orderId` to every log line in this thread
- → `structured.format.console` writes one JSON object per line

APPLICATION.YML

```
logging:
  level:
    root: INFO
    com.example: DEBUG
    org.hibernate.SQL: DEBUG
  file.name: logs/shop.log
  structured.format.console: ecs      # JSON: ecs, logstash or gelf
```

LEVELS

LEVEL	USE FOR
ERROR	Failures needing action
WARN	Unexpected, handled
INFO	Key business events
DEBUG	Diagnostics

LOGGING.*

PROPERTY	SETS
`level.<pkg>`	Level per package
`file.name`	Log file
`logback.rollingpolicy.*`	Rotation
`group.<name>`	Named groups

LOGBACK-SPRING.XML

ELEMENT	DOES
`<springProfile>`	Per profile
`<springProperty>`	Read a property
`<include>`	Boot defaults

⚠ COMMON MISTAKES

- String concatenation in log calls; it runs even when DEBUG is off
- `log.error(e.getMessage())` ; the stack trace is lost
- Logging passwords, tokens or full request bodies

✓ TIPS & BEST PRACTICES

- Change levels live via `/actuator/loggers` while debugging
- Use structured logs in production; plain text in dev
- Use `logging.group.web` style groups for related packages

QUICK SUMMARY

- SLF4J + Logback by default
- `logging.level.pkg=DEBUG`
- `structured.format.console` : JSON
- `MDC` : per-request context

Application Events

APPLICATION EVENTS

Beans can communicate through **events** without depending on each other: one publishes with **`ApplicationEventPublisher`**, others react with **`@EventListener`** methods. Listeners run **synchronously** in the same thread by default. **`@TransactionalEventListener`** waits until the surrounding transaction **commits**, which is the right place for emails and messages.

NOTES

- Any object can be an event; records work well
- Add `@Async` to a listener to run it on another thread
- Boot publishes lifecycle events such as `ApplicationReadyEvent`
- Spring Modulith builds on events for modular apps

PUBLISH AND LISTEN

```
public record OrderPlaced(long orderId, String email) { }

@Service
class OrderService {
    private final ApplicationEventPublisher events;
    OrderService(ApplicationEventPublisher events) { this.events = events; }

    @Transactional
    public void place(Order o) {
        repo.save(o);
        events.publishEvent(new OrderPlaced(o.id(), o.email()));
    }
}

@Component
class OrderListeners {
    @EventListener
    void updateStats(OrderPlaced e) { … }                  // same tx, sync

    @TransactionalEventListener                            // after commit
    void sendEmail(OrderPlaced e) { mail.confirm(e.email()); }
}
```

CODE EXPLAINED

- → `publishEvent` calls each synchronous listener before returning
- → A rollback means `sendEmail` never runs
- → `@EventListener(condition = "#e.orderId > 1000")` filters events
- → Listener exceptions in sync listeners propagate to the publisher

TRANSACTIONPHASE

PHASE	RUNS
`AFTER_COMMIT`	After commit (default)
`AFTER_ROLLBACK`	After rollback
`AFTER_COMPLETION`	Either
`BEFORE_COMMIT`	Just before commit

BOOT LIFECYCLE EVENTS

EVENT	WHEN
`ApplicationStartingEvent`	Very start
`ApplicationStartedEvent`	Context refreshed
`ApplicationReadyEvent`	Ready for traffic
`ApplicationFailedEvent`	Startup failed

@EVENTLISTENER OPTIONS

OPTION	DOES
`classes`	Listen to several types
`condition`	SpEL filter
`@Order`	Order listeners
`return value`	Publishes it as event

⚠ COMMON MISTAKES

- Sending emails in a plain `@EventListener`, then the transaction rolls back
- Writing to the DB in an `AFTER_COMMIT` listener without a new transaction
- Events for everything; direct calls are clearer inside one module

✓ TIPS & BEST PRACTICES

- Use `REQUIRES_NEW` for DB writes in after-commit listeners
- Combine `@Async` + `@TransactionalEventListener` for slow side effects
- Test with `@RecordApplicationEvents` + `ApplicationEvents`

QUICK SUMMARY

- `publishEvent(obj)` : fire
- `@EventListener` : react (sync)
- `@TransactionalEventListener` : after commit
- `@Async` : off the caller's thread

Scheduling

RUNNING TASKS ON A SCHEDULE

Add **`@EnableScheduling`** and annotate methods with **`@Scheduled`** to run them periodically: at a **fixed rate**, with a **fixed delay** between runs, or on a **cron** expression. Methods take no arguments and return `void`. By default all scheduled tasks share **one thread**, so a slow task delays the others unless you raise the pool size or use virtual threads.

NOTES

- Spring cron has **6 fields**: second first
- `zone` sets the cron time zone (default: server's)
- Values can come from properties: `cron = "${jobs.cleanup}"`
- Multiple instances each run the job; use ShedLock to run once

SCHEDULED JOBS

```
@Configuration
@EnableScheduling
class SchedulingConfig { }

@Component
class Jobs {

    @Scheduled(fixedRate = 30, timeUnit = TimeUnit.SECONDS)
    void pollFeed() { … }                    // every 30 s, start to start

    @Scheduled(fixedDelay = 5000, initialDelay = 10000)
    void drainQueue() { … }                  // 5 s after previous ends

    @Scheduled(cron = "0 0 2 * * *", zone = "America/Chicago")
    void nightlyCleanup() { … }              // 02:00 every day

    @Scheduled(cron = "0 */15 9-17 * * MON-FRI")
    void businessHours() { … }               // every 15 min, 9-5 weekdays

    @Scheduled(cron = "${jobs.report.cron:0 0 6 * * MON}")
    void weeklyReport() { … }                // from config
}
```

CODE EXPLAINED

- → `fixedRate` measures from the start of each run
- → `fixedDelay` waits after the previous run finishes
- → `initialDelay` postpones the first run
- → Cron `"-"` disables a job (handy via properties)

THREAD POOL

```
spring.task.scheduling.pool.size=4
spring.task.scheduling.thread-name-prefix=sched-
```

CRON FIELDS

POS	FIELD
1	Second (0–59)
2	Minute (0–59)
3	Hour (0–23)
4	Day of month
5 / 6	Month / weekday

CRON EXAMPLES

EXPRESSION	RUNS
`0 * * * * *`	Every minute
`0 0 * * * *`	Every hour
`0 0 0 * * *`	Midnight
`0 0 9 * * MON`	Mondays 9:00
`@daily`	Midnight (macro)

@SCHEDULED ATTRIBUTES

ATTRIBUTE	UNIT
`fixedRate`	`timeUnit` (ms)
`fixedDelay`	`timeUnit` (ms)
`initialDelay`	`timeUnit` (ms)
`cron + zone`	Expression

⚠ COMMON MISTAKES

- Unix 5-field cron (`0 2 * * *`); Spring expects 6 fields
- Long jobs on the single default thread, blocking every other job
- Every replica running the same job at once in a cluster

✓ TIPS & BEST PRACTICES

- Use ShedLock or a database lock for cluster-wide single runs
- With virtual threads enabled, each run gets its own virtual thread
- Keep job methods thin; call a service you can test directly

QUICK SUMMARY

- `@EnableScheduling` + `@Scheduled`
- `fixedRate` vs `fixedDelay`
- 6-field cron + `zone`
- `pool.size`: parallel jobs

@Async & Virtual Threads

ASYNC WORK AND VIRTUAL THREADS

`@Async` runs a method on a **task executor** instead of the caller's thread; it returns `void` or a **`CompletableFuture`**. Enable it with **`@EnableAsync`**. On Java 21+, setting **`spring.threads.virtual.enabled=true`** switches Tomcat, `@Async`, `@Scheduled` and other executors to **virtual threads**, so blocking code scales without reactive APIs.

NOTES

- Boot's executor is `applicationTaskExecutor`
- `@Async` works through a proxy: no self-calls
- Exceptions from `void` methods go to an `AsyncUncaughtExceptionHandler`
- Security and MDC context don't cross threads by default

@ASYNC AND FUTURES

```
@Configuration
@EnableAsync
class AsyncConfig { }

@Service
class ReportService {

    @Async
    public CompletableFuture<Report> build(long id) {
        return CompletableFuture.completedFuture(generate(id));    // slow work
    }

    @Async
    public void emailLater(String to) { mail.send(to); }          // fire & forget
}

// caller: run three in parallel, then wait for all
var a = reports.build(1);
var b = reports.build(2);
var c = reports.build(3);
CompletableFuture.allOf(a, b, c).join();
List<Report> all = Stream.of(a, b, c).map(CompletableFuture::join).toList();
```

CODE EXPLAINED

- → Each `build()` call returns immediately with a future
- → `allOf(...).join()` waits until all three finish
- → Return `CompletableFuture` so callers see exceptions
- → With virtual threads, `@Async` uses a new virtual thread per task

TURN ON VIRTUAL THREADS

```
spring.threads.virtual.enabled=true     # Java 21+
```

@ASYNC RETURN TYPES

TYPE	CALLER GETS
`void`	Nothing
`CompletableFuture<T>`	Result later
`Future<T>`	Older API

SPRING.TASK.EXECUTION.*

PROPERTY	SETS
`pool.core-size`	Base threads
`pool.max-size`	Max threads
`pool.queue-capacity`	Queue size
`thread-name-prefix`	Thread names

VIRTUAL THREADS

GOOD FOR	WATCH OUT
`Blocking I/O`	CPU-heavy work
`Many requests`	`synchronized` pinning
`Simple code`	Unbounded concurrency

⚠ COMMON MISTAKES

- Calling an `@Async` method from the same class; it runs synchronously
- `void` async methods that fail silently; return a future or add a handler
- Unlimited parallel calls to a downstream service on virtual threads

✓ TIPS & BEST PRACTICES

- Cap concurrency with `@ConcurrencyLimit` (Sheet 64)
- Copy MDC/security context with a `TaskDecorator`
- Use `@Async("executorName")` to pick a dedicated pool

QUICK SUMMARY

- `@EnableAsync` + `@Async`
- `CompletableFuture<T>`: async result
- `virtual.enabled=true`: Java 21+
- Proxy rules: no self-calls

Caching

CACHING METHOD RESULTS

The **cache abstraction** stores method results by their arguments: **`@Cacheable`** returns a cached value when present, **`@CachePut`** always runs and updates the cache, **`@CacheEvict`** removes entries. Enable it with **`@EnableCaching`**. Boot picks a provider from the classpath: a simple map by default, or **Caffeine**, **Redis** and others.

NOTES

- Add `spring-boot-starter-cache` plus a provider
- The key defaults to the method arguments
- Cached values for Redis must be serializable
- Proxy-based: self-calls skip the cache

CACHE ANNOTATIONS

```
@Configuration @EnableCaching class CacheConfig { }

@Service
class ProductService {

    @Cacheable("products")                          // key = id
    public Product find(long id) { return repo.findById(id).orElseThrow(); }

    @Cacheable(value = "search", key = "#q.toLowerCase()",
               unless = "#result.isEmpty()")
    public List<Product> search(String q) { … }

    @CachePut(value = "products", key = "#p.id")     // refresh entry
    public Product update(Product p) { return repo.save(p); }

    @CacheEvict(value = "products", key = "#id")
    public void delete(long id) { repo.deleteById(id); }
}
```

CODE EXPLAINED

- → Second `find(7)` call returns the cached product, no query
- → `unless` skips caching empty results
- → `@CachePut` keeps the cache in sync after updates
- → `@CacheEvict(allEntries = true)` clears a whole cache

PROVIDER CONFIG

```
spring.cache:
  cache-names: products, search
  caffeine.spec: maximumSize=1000,expireAfterWrite=10m
  # type: redis   +   spring.cache.redis.time-to-live: 10m
```

ANNOTATIONS

ANNOTATION	DOES
`@Cacheable`	Read-through
`@CachePut`	Always run, store
`@CacheEvict`	Remove entries
`@Caching`	Combine several
`@CacheConfig`	Class defaults

ATTRIBUTES

ATTRIBUTE	PURPOSE
`key`	SpEL key
`condition`	Cache if true (before)
`unless`	Skip if true (after)
`sync`	One loader per key

PROVIDERS

TYPE	BEST FOR
`simple`	Dev (no expiry)
`caffeine`	Local, fast, TTL
`redis`	Shared across nodes
`none`	Disable in tests

⚠ COMMON MISTAKES

- Caching mutable entities, then changing them; the cache holds the same object
- No expiry or size limit; the cache grows until memory runs out
- Expecting `@Cacheable` to work on a call from the same class

✓ TIPS & BEST PRACTICES

- Use `sync = true` to stop a cache stampede on cold keys
- Watch `cache.gets` hit/miss metrics in Actuator
- Set `spring.cache.type=none` in tests that need fresh data

QUICK SUMMARY

- `@EnableCaching`
- `@Cacheable` : read-through
- `@CacheEvict` / `@CachePut`
- Caffeine local · Redis shared

Retry & Resilience

RETRIES AND CONCURRENCY LIMITS

Spring Framework 7 brings resilience into the core: **`@Retryable`** re-runs a method when it throws, with configurable back-off and jitter, and **`@ConcurrencyLimit`** caps how many callers run a method at once. Turn both on with **`@EnableResilientMethods`**. For code-level control, **`RetryTemplate`** with a `RetryPolicy` does the same programmatically.

NOTES

- Annotations live in `org.springframework.resilience`
- Defaults: any exception, `maxRetries = 3`, 1 s delay
- Total attempts = 1 + `maxRetries`
- Replaces the separate Spring Retry project for most uses

ANNOTATIONS

```
@Configuration @EnableResilientMethods class ResilienceConfig { }

@Service
class InventoryClient {

    @Retryable(includes = ResourceAccessException.class,
               maxRetries = 4, delay = 200, multiplier = 2,
               maxDelay = 2000, jitter = 50)
    public Stock check(String sku) {
        return http.get().uri("/stock/{sku}", sku).retrieve().body(Stock.class);
    }

    @ConcurrencyLimit(10)              // at most 10 calls in flight
    public Report heavyReport() { … }
}
```

RETRYTEMPLATE

```
var policy = RetryPolicy.builder()
    .includes(IOException.class)
    .maxRetries(3)
    .delay(Duration.ofMillis(100))
    .multiplier(2)
    .build();

var template = new RetryTemplate(policy);
String body = template.invoke(() -> download(url));   // rethrows last error
```

CODE EXPLAINED

- → Delays grow 200, 400, 800, 1600 ms, capped at 2 s
- → Only `ResourceAccessException` (I/O errors) is retried
- → `@ConcurrencyLimit` makes extra callers wait
- → `execute()` instead of `invoke()` throws `RetryException` with all attempts

@RETRYABLE ATTRIBUTES

ATTRIBUTE	MEANING
`includes / value`	Retry these
`excludes`	Never retry these
`maxRetries`	Extra attempts
`delay / maxDelay`	Back-off (ms)

RETRY WHEN…

ERROR	RETRY?
`Timeout / 503`	Yes
`Connection reset`	Yes
`400 / validation`	No
`Non-idempotent POST`	Careful

RELATED OPTIONS

TOOL	USE
`RetryTemplate`	Programmatic
`MethodRetryEvent`	Log each failure
`Resilience4j`	Circuit breakers

⚠ COMMON MISTAKES

- Retrying everything, including bad requests that will never succeed
- Retrying non-idempotent operations and charging a card twice
- Calling the `@Retryable` method from the same class; no retries happen

✓ TIPS & BEST PRACTICES

- Always set timeouts before adding retries
- Add jitter so many clients don't retry in lockstep
- Use `@ConcurrencyLimit` to protect downstream services on virtual threads

QUICK SUMMARY

- `@EnableResilientMethods`
- `@Retryable(maxRetries, delay…)`
- `@ConcurrencyLimit(n)`
- `RetryTemplate` + `RetryPolicy`

WebFlux & Reactive Basics

REACTIVE WEB WITH WEBFLUX

Spring WebFlux is the non-blocking alternative to Spring MVC, running on **Netty** by default. Handlers return **`Mono<T>`** (0–1 item) or **`Flux<T>`** (0–N items) from **Project Reactor**, and nothing runs until the framework **subscribes**. Use it for streaming and very high concurrency with reactive drivers; with virtual threads, Spring MVC now covers most other cases.

NOTES

- Add `spring-boot-starter-webflux` (not together with webmvc as server)
- Every layer must be non-blocking: R2DBC, reactive Mongo/Redis
- Same annotations as MVC: `@RestController`, `@GetMapping`
- Or use functional `RouterFunction` endpoints

ANNOTATED AND FUNCTIONAL ENDPOINTS

```
@RestController
@RequestMapping("/api/products")
class ProductController {
    private final ProductRepository repo;          // ReactiveCrudRepository

    @GetMapping("/{id}")
    Mono<Product> get(@PathVariable String id) {
        return repo.findById(id)
            .switchIfEmpty(Mono.error(new ProductNotFound(id)));
    }

    @GetMapping(produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    Flux<Product> stream() {                        // server-sent events
        return repo.findAll().delayElements(Duration.ofMillis(100));
    }

    @PostMapping
    Mono<Product> create(@RequestBody Mono<Product> body) {
        return body.map(Product::normalize).flatMap(repo::save);
    }
}
```

CODE EXPLAINED

- → `switchIfEmpty` turns an empty `Mono` into an error (404)
- → `TEXT_EVENT_STREAM` streams items as they arrive
- → `flatMap` chains another async call (the save)
- → `RouterFunction` beans are the annotation-free alternative

REACTOR OPERATORS

OPERATOR	DOES
`map`	Transform sync
`flatMap`	Transform async
`filter`	Keep matching
`zip`	Combine sources
`onErrorResume`	Fallback

MVC VS WEBFLUX

ASPECT	MVC / WEBFLUX
`Server`	Tomcat / Netty
`Model`	Blocking / Non-blocking
`Data`	JDBC, JPA / R2DBC
`Testing`	MockMvc / WebTestClient

CREATING PUBLISHERS

FACTORY	EMITS
`Mono.just(x)`	One value
`Mono.empty()`	Nothing
`Flux.fromIterable`	List items
`Flux.interval(d)`	Ticks

⚠ COMMON MISTAKES

- Calling JDBC/JPA or `block()` inside a WebFlux handler; it stalls the event loop
- Building a `Mono` and never returning or subscribing to it
- Choosing WebFlux for a CRUD app that virtual threads would serve fine

✓ TIPS & BEST PRACTICES

- Use `publishOn(Schedulers.boundedElastic())` for unavoidable blocking calls
- Test with `WebTestClient` and Reactor's `StepVerifier`
- Turn on `BlockHound` in tests to catch blocking calls

QUICK SUMMARY

- `Mono` (0–1) · `Flux` (0–N)
- Same annotations, reactive types
- Non-blocking end to end
- Lazy: runs on subscribe

Packaging & Containers

SHIPPING THE APP

The Boot plugin builds an **executable jar** you run with `java -jar`. For containers, **`build-image`** / **`bootBuildImage`** creates an OCI image with **Cloud Native Buildpacks** (no Dockerfile needed), or you write a Dockerfile that uses Boot's **layered** jar so dependency layers are cached between builds. Shutdown is **graceful** by default: in-flight requests finish first.

NOTES

- Configure via env vars: `SPRING_PROFILES_ACTIVE`, `SERVER_PORT`
- Buildpacks pick a JRE and tune memory for the container
- Layers: dependencies → loader → snapshot deps → application
- `server.shutdown=immediate` turns graceful shutdown off

BUILD COMMANDS

```
./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=shop:1.4
./gradlew bootBuildImage --imageName=shop:1.4
docker run -p 8080:8080 -e SPRING_PROFILES_ACTIVE=prod shop:1.4
```

DOCKERFILE WITH LAYERS

```
FROM eclipse-temurin:25-jre AS build
WORKDIR /app
COPY target/shop.jar app.jar
RUN java -Djarmode=tools -jar app.jar extract --layers --launcher

FROM eclipse-temurin:25-jre
WORKDIR /app
COPY --from=build /app/app/dependencies/ ./
COPY --from=build /app/app/spring-boot-loader/ ./
COPY --from=build /app/app/snapshot-dependencies/ ./
COPY --from=build /app/app/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

CODE EXPLAINED

- → `imageName` tags the image; push it to any registry
- → `extract --layers` splits the jar into cacheable folders
- → Code changes rebuild only the small `application` layer
- → `JarLauncher` starts the app from the extracted layout

DEPLOYMENT PROPERTIES

PROPERTY	SETS
`server.port`	HTTP port
`server.shutdown`	`graceful` (default)
`…shutdown-phase`	Grace period
`…forward-headers…`	Behind a proxy

KUBERNETES

SETTING	VALUE
`livenessProbe`	`…/liveness`
`readinessProbe`	`…/readiness`
`preStop sleep`	Drain traffic
`Config`	ConfigMap → env vars

JVM FLAGS

FLAG	EFFECT
`-XX:MaxRAMPercentage=75`	Heap vs container
`-XX:+UseZGC`	Low-pause GC
`-Xss512k`	Smaller stacks

⚠ COMMON MISTAKES

- Fixed `-Xmx` larger than the container limit; the pod gets OOM-killed
- Copying the fat jar as a single layer; every build re-uploads everything
- Baking secrets into the image instead of injecting env vars

✓ TIPS & BEST PRACTICES

- Use buildpacks unless you need full control of the Dockerfile
- Run as a non-root user in custom images
- Use CDS or AOT cache (Java 25) to cut startup time

QUICK SUMMARY

- `java -jar`: run anywhere
- `bootBuildImage`: OCI image
- `extract --layers`: Docker caching
- Graceful shutdown by default

GraalVM Native Images

native-maven-plugin · nativeCompile · AOT processing · RuntimeHints · reflection config · startup / memory tradeoffs

NATIVE EXECUTABLES

GraalVM Native Image compiles the app ahead of time into a standalone executable that starts in milliseconds and uses far less memory, at the cost of long builds and a **closed world**: everything reachable must be known at build time. Spring's **AOT processing** generates the bean setup in advance; you add **runtime hints** for reflection, resources or proxies Spring can't detect.

NOTES

- Boot 4 needs **GraalVM 25** or later
- Profiles and `@Conditional` beans are fixed at build time
- Run tests natively with `-PnativeTest` / `nativeTest`
- Buildpacks can build a native image without local GraalVM

BUILD COMMANDS

```
./mvnw -Pnative native:compile          # target/shop (executable)
./gradlew nativeCompile                 # build/native/nativeCompile/shop
./mvnw -Pnative spring-boot:build-image # native container image
./target/shop                           # starts in ~50 ms
```

RUNTIME HINTS

```
@Configuration
@ImportRuntimeHints(ShopHints.class)
@RegisterReflectionForBinding(LegacyDto.class)   // JSON via reflection
class NativeConfig { }

class ShopHints implements RuntimeHintsRegistrar {
    @Override
    public void registerHints(RuntimeHints hints, ClassLoader cl) {
        hints.resources().registerPattern("templates/*.txt");
        hints.reflection().registerType(PluginImpl.class,
            MemberCategory.INVOKE_PUBLIC_CONSTRUCTORS);
        hints.proxies().registerJdkProxy(Auditor.class);
    }
}
```

CODE EXPLAINED

- → Maven's `native` profile comes from `spring-boot-starter-parent`
- → `@RegisterReflectionForBinding` covers Jackson-bound types
- → Resource patterns include files read with `getResource()`
- → Missing hints fail at **runtime**, so test the native binary

GRADLE PLUGIN

```
plugins { id("org.graalvm.buildtools.native") }   // version: use Initializr's
```

JVM VS NATIVE

ASPECT	JVM / NATIVE
Startup	Seconds / ms
Memory	Higher / lower
Build time	Fast / minutes
Peak throughput	Higher / lower
Reflection	Free / needs hints

HINT TYPES

API	COVERS
`reflection()`	Classes, methods
`resources()`	Classpath files
`serialization()`	Java serialization
`proxies()`	JDK proxies

GOOD FIT?

WORKLOAD	NATIVE?
Serverless / CLI	Yes
Scale-to-zero	Yes
Long-running, hot	JVM often wins
Heavy reflection libs	Painful

⚠ COMMON MISTAKES

- Testing only on the JVM; reflection gaps show up only in the native binary
- Choosing beans with runtime profiles; native fixes them at build time
- Expecting native to be faster at peak load; the JIT often wins there

✓ TIPS & BEST PRACTICES

- Run the GraalVM tracing agent to discover needed hints
- Build native images in CI; they take minutes and lots of RAM
- Try Java 25's AOT cache first if you only need faster startup

QUICK SUMMARY

- `native:compile` / `nativeCompile`
- AOT: beans prepared at build time
- `RuntimeHintsRegistrar`: extra hints
- Fast start, closed world